

Departamento de Electrónica e Computación
Escuela Técnica Superior de Ingeniería



TRABAJO FIN DE MÁSTER
MÁSTER INTERUNIVERSITARIO EN
COMPUTACIÓN DE ALTAS PRESTACIONES

Desarrollo y evaluación de aplicaciones paralelas con OMP4Py

Estudiante: Alejandro Vedo Godines
Director/a/es/as: César Alfredo Piñeiro Pomar
Juan Carlos Pichel Campos

Santiago de Compostela, junio de 2026.

Agradecimientos

A mis tutores, César Alfredo Piñeiro Pomar y Juan Carlos Pichel Campos, por la orientación técnica durante el desarrollo del trabajo y por facilitar el contexto del proyecto OMP4Py.

Resumen

OMP4Py lleva el modelo de directivas de OpenMP al código Python. Este trabajo lo pone a prueba adaptando cinco NAS Parallel Benchmarks de naturaleza distinta (EP, CG, IS, MG y FT) y ejecutándolos en el supercomputador Finisterrae III con Python 3.15.0b1t *free-threaded*.

La evaluación cubre corrección, tiempo absoluto y escalabilidad en los cuatro modos de OMP4Py, desde el intérprete puro hasta el Cython tipado. La campaña principal con Python 3.15.0b1t reúne 630 ejecuciones, y la comparación entre Python 3.13.13t, 3.14.4t y 3.15.0b1t añade otras 2340, en modos interpretados y compilados. Todas verificaron correctamente.

El factor más determinante es el tipado Cython. En clase W, al pasar del modo compilado sin tipos al tipado los tiempos caen en todos los benchmarks, desde el orden de $15\times$ en FT hasta más de dos órdenes de magnitud en los casos más favorables. La causa es directa. Sin anotaciones, cada operación del bucle crítico arrastra el despacho de objetos Python. Con ellas, el modo 3 es la única configuración viable para cargas grandes.

La escalabilidad entre hilos es más desigual y depende del benchmark. EP y FT aceleran con claridad en clase A y superan $10\times$ de speedup. Los otros tres no escalan de forma sostenida. El diagnóstico posterior atribuye esa diferencia a sincronización, granularidad y gestión de equipos de hilos, combinando perf stat con el muestreo y trazado de Python 3.15 en Finisterrae III. La versión del intérprete apenas influye en los modos compilados. En ejecución interpretada, en cambio, Python 3.13t escala bastante peor al aumentar los hilos, salvo en EP, una penalización que el tipado elimina. Con la afinidad de hilos corregida, Python 3.13t, 3.14t y 3.15t rinden casi igual en EP y FT. OMP4Py resulta útil en la práctica, pero su rendimiento no es automático y depende del tipado Cython, de la configuración del runtime y de la estructura de cada kernel.

Abstract

OMP4Py brings OpenMP's directive model to plain Python. This work puts it to the test by adapting five NAS Parallel Benchmarks of different nature (EP, CG, IS, MG, and FT) and running them on the Finisterrae III supercomputer with Python 3.15.0b1t free-threaded.

The evaluation covers correctness, absolute time, and scalability across OMP4Py's four execution modes, from the pure interpreter to typed Cython. The main Python 3.15.0b1t campaign gathers 630 runs, and the comparison across Python 3.13.13t, 3.14.4t, and 3.15.0b1t adds another 2340, spanning interpreted and compiled modes. All of them verified correctly.

By far the dominant factor is Cython typing. In class W, moving from the untyped compiled mode to the typed one reduces the times in every benchmark, from around $15\times$ in FT to more than two orders of magnitude in the most favorable cases. The reason is straightforward. Without annotations, every operation in the critical loop drags Python object dispatch along. With them, mode 3 becomes the only viable configuration for large workloads.

Thread scalability is more uneven and depends on the benchmark. EP and FT speed up clearly in class A, exceeding $10\times$ speedup. The other three do not show sustained scalability. The later diagnosis attributes this difference to synchronization, granularity, and thread-team management, combining perf stat with Python 3.15 sampling and tracing on Finisterrae III. The interpreter version barely matters in the compiled modes. In interpreted execution, by contrast, Python 3.13t scales noticeably worse as threads grow, except on EP, a penalty that typing removes. Once thread affinity is fixed, Python 3.13t, 3.14t, and 3.15t behave almost identically on EP and FT. OMP4Py is useful in practice, but its performance is not automatic. It depends on Cython typing, on the runtime configuration, and on the structure of each kernel.

Palabras clave:

- OMP4Py
- OpenMP
- Python free-threaded
- NAS Parallel Benchmarks
- Computación de altas prestaciones
- Cython
- Escalabilidad

Keywords:

- OMP4Py
- OpenMP
- Free-threaded Python
- NAS Parallel Benchmarks
- High Performance Computing
- Cython
- Scalability

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Contribuciones	3
1.4. Alcance	3
1.5. Estructura de la memoria	4
2. Contexto tecnológico	5
2.1. Python y el problema del GIL	5
2.2. OMP4Py	5
2.3. NAS Parallel Benchmarks	7
2.4. Trabajo relacionado	7
3. Adaptación de los NAS Parallel Benchmarks a OMP4Py	11
3.1. Benchmarks adaptados	11
3.2. Metodología de adaptación	12
3.3. Decisiones de implementación	13
3.3.1. Separación de kernels y orquestación	13
3.3.2. Compilación dinámica tras inicialización	13
3.3.3. Indexado directo en NumPy	13
3.3.4. Variables privadas explícitas	13
3.3.5. Funciones auxiliares compilables localmente	13
3.4. Particularidades por benchmark	14
3.4.1. EP	14
3.4.2. CG	14
3.4.3. IS	15
3.4.4. MG	15
3.4.5. FT	16

4. Metodología experimental	17
4.1. Objetivo de la evaluación	17
4.2. Entorno de ejecución	17
4.3. Automatización de la campaña	18
4.4. Diseño de la matriz experimental	19
4.4.1. Campaña principal en clase S	19
4.4.2. Campaña principal en clase W	19
4.4.3. Campaña principal en clase A	19
4.4.4. Campaña comparativa entre versiones	19
4.4.5. Campaña de profiling	20
4.5. Justificación de la restricción de modos en clases grandes	20
4.6. Métricas y tratamiento de los resultados	21
4.7. Amenazas a la validez	22
4.7.1. Validez interna	22
4.7.2. Validez de medida	23
4.7.3. Validez externa	23
4.7.4. Validez del instrumento	23
5. Resultados	25
5.1. Efecto del modo de ejecución en clase S	25
5.2. Escalabilidad en clase S	27
5.3. Modos compilados en clase W	28
5.4. Evaluación de escalabilidad en clase A	30
5.5. Síntesis global del modo 3	31
5.6. Curvas por benchmark	32
5.7. Limitaciones de la medida	34
5.8. Conclusiones del capítulo	34
6. Profiling y diagnóstico	35
6.1. Objetivo del profiling	35
6.2. Control de entorno y afinidad	35
6.3. Diagnóstico común con perf stat	36
6.4. Interpretación conjunta	37
6.5. Profiling con el nuevo sistema de Python 3.15	38
6.5.1. Entorno y metodología	38
6.5.2. Del muestreo a las tablas	38
6.5.3. Coste de sincronización	39
6.5.4. Gestión de hilos y balance de carga	39

6.5.5.	Recuento exacto con el profiler de trazado	41
6.5.6.	Síntesis por benchmark	41
6.6.	Implicaciones para OMP4Py	42
6.6.1.	Reutilización del equipo de hilos	42
6.6.2.	Coste de la barrera y la reducción	43
6.6.3.	Calidad del código compilado	43
7.	Comparación entre versiones de Python <i>free-threaded</i>	45
7.1.	Diseño de la campaña	45
7.2.	Efecto de la versión en los modos interpretados	46
7.3.	El tipado neutraliza el efecto de versión	49
7.4.	Conclusiones	51
8.	Conclusiones	53
8.1.	Hallazgos principales	53
8.1.1.	Cobertura funcional del port	53
8.1.2.	El tipado Cython como factor dominante	53
8.1.3.	Escalabilidad de EP y FT	54
8.1.4.	Límites de CG, IS y MG	54
8.1.5.	El tipado neutraliza la versión de Python	54
8.1.6.	La afinidad del entorno como factor crítico	54
8.1.7.	Escalabilidad real frente a paralelismo ineficiente	55
8.2.	Limitaciones	55
8.3.	Trabajo futuro	55
8.3.1.	Instrumentación y campañas complementarias	55
8.3.2.	Mejoras del runtime	56
8.3.3.	Mejoras por benchmark	56
8.3.4.	Extensión a BT, SP y LU	56
8.4.	Valoración global	57
	Apéndices	59
A.	Interfaz de ejecución de los benchmarks	61
B.	Kernel EP adaptado en modo tipado	63
C.	Infraestructura de ejecución en Finisterrae III	65
D.	Estructura de los resultados de campaña	67

E. Material completo de la comparación entre versiones en modo 3	69
F. Glosario de acrónimos	77
G. Glosario de términos	79
Bibliografía	81

Índice de figuras

2.1. Flujo de ejecución de un programa OMP4Py. Las directivas se procesan mediante transformación AST en todos los modos. El modo determina si los kernels del usuario se compilan con Cython y qué implementación del runtime ejecuta la distribución de trabajo y la sincronización.	6
4.1. Flujo de la infraestructura de campaña en Finisterrae III. Generación de la matriz de combinaciones, envío como array de Slurm, ejecución de cada combinación y consolidación de resultados.	18
5.1. Tiempo NAS con un hilo en clase S para los cuatro modos de ejecución. El eje vertical usa escala logarítmica.	26
5.2. Clase S, modo 3, Python 3.15t. Speedup y tiempo NAS por número de hilos. Cada línea corresponde a un benchmark.	27
5.3. Clase W, modo 2, Python 3.15t. Speedup y tiempo NAS por número de hilos.	29
5.4. Clase W, modo 3, Python 3.15t. Speedup y tiempo NAS por número de hilos. Cada línea corresponde a un benchmark.	29
5.5. Clase A, modo 3, Python 3.15t. Speedup y tiempo NAS por número de hilos. Cada línea corresponde a un benchmark.	31
5.6. Mapa de calor del speedup a 32 hilos en modo 3, Python 3.15t, para las clases S, W y A.	32
5.7. Speedup por clase (S, W, A) en modo 3, Python 3.15t, para cada benchmark.	33
7.1. Tiempo NAS por número de hilos en modo 1 interpretado, clase S, para Python 3.13t, 3.14t y 3.15t. En CG, FT, IS y MG la 3.13t se dispara al aumentar los hilos, mientras que EP, <i>embarrassingly parallel</i> , mantiene las tres versiones juntas.	48

7.2. Tiempo NAS por número de hilos en modo 3 compilado, clase W, para Python 3.13t, 3.14t y 3.15t. Las tres versiones se superponen en todos los benchmarks, en claro contraste con la divergencia del modo interpretado (Figura 7.1).	50
E.1. Tiempo NAS con 32 hilos en modo 3, clases S y W, para las tres versiones de Python. El eje vertical usa escala logarítmica.	73
E.2. EP, modo 3. Speedup por versión de Python en las clases S y W.	73
E.3. FT, modo 3. Speedup por versión de Python en las clases S y W.	74
E.4. CG, modo 3. Speedup por versión de Python en las clases S y W.	74
E.5. IS, modo 3, por versión de Python. En clase S se muestra el tiempo NAS porque el speedup no es fiable (tiempo con un hilo redondeado a 0.00 s). En clase W se muestra el speedup.	74
E.6. MG, modo 3. Speedup por versión de Python en las clases S y W.	75

Índice de tablas

1.1. Objetivos específicos del trabajo.	2
2.1. Relación entre OMP4Py y otros enfoques de paralelismo o aceleración en Python.	8
3.1. Estado de los benchmarks NAS en el repositorio.	12
4.1. Matriz experimental. Todas las configuraciones ejecutan CG, EP, FT, IS y MG con 1, 2, 4, 8, 16 y 32 hilos.	20
4.2. Estabilidad de las repeticiones válidas. CV, rango y diferencia mediana/mejor se calculan por combinación con mediana NAS mayor que cero.	21
5.1. Cobertura de las campañas finales ejecutadas en Finisterrae III.	25
5.2. Mejor tiempo NAS en clase S con un hilo para los cuatro modos de ejecución en Python 3.15t.	26
5.3. Speedup a 32 hilos respecto a 1 hilo en clase S para cada modo de Python 3.15t.	27
5.4. Mejor tiempo NAS en clase W con un hilo para los modos compilados de Python 3.15t.	28
5.5. Speedup a 32 hilos respecto a 1 hilo en clase W para los modos compilados de Python 3.15t.	29
5.6. Mejor tiempo NAS en clase A, modo 3, Python 3.15t.	30
5.7. Speedup por número de hilos en clase A, modo 3, Python 3.15t.	30
5.8. Mejor número de hilos, speedup y rendimiento (Mop/s) a 32 hilos en modo 3, Python 3.15t.	31
6.1. Perfil homogéneo con perf stat en Finisterrae III: Python 3.15t, clase W, modo 3 y recuentos de 1, 8 y 32 hilos para todos los benchmarks.	37
6.2. Coste de sincronización medido con profiling.sampling en Finisterrae III. Porcentaje de muestras de espera sobre ejecución paralela; clase S, modo 1.	39

6.3.	Gestión de hilos medida con <code>profiling.sampling</code> en Finisterrae III. Hilos trabajadores distintos observados con 16 hilos; clase S, modo 1.	40
6.4.	Recuentos exactos del runtime medidos con <code>profiling.tracing</code> en Finisterrae III. Clase S, modo 1 y 8 hilos.	41
7.1.	Tiempo NAS por número de hilos en los modos interpretados 0 y 1, clase S, para Python 3.13t, 3.14t y 3.15t. Tiempos en segundos. Al crecer el número de hilos, Python 3.13t se degrada mucho más que 3.14t y 3.15t en CG, FT, IS y MG. EP, <i>embarrassingly parallel</i> , es la excepción.	46
7.2.	Tiempo NAS con un hilo en los modos compilados 2 y 3, clases S y W, para Python 3.13t, 3.14t y 3.15t. El paso de modo 2 a modo 3 (tipado) acelera entre 15× y más de 400× según el benchmark, de forma casi idéntica en las tres versiones.	49
E.1.	Mejor tiempo NAS por número de hilos en modo 3 para cada versión de Python.	70
E.2.	Speedup por número de hilos en modo 3 para cada versión de Python.	71
E.3.	Mejor número de hilos en modo 3 para cada versión, clase y benchmark.	72
E.4.	Versión más rápida con 32 hilos en modo 3 y margen sobre la segunda mejor.	72

Introducción

PYTHON ocupa una posición consolidada en la computación científica. Su ecosistema de bibliotecas, la legibilidad del código y la rapidez de prototipado lo han convertido en el lenguaje de referencia para análisis numérico y aprendizaje automático [1]. Sin embargo, la ejecución paralela con múltiples hilos ha estado históricamente limitada por el *Global Interpreter Lock* (GIL), que impide la ejecución simultánea de bytecode Python en un mismo proceso. Las versiones *free-threaded* de Python, disponibles experimentalmente a partir de Python 3.13 [2], eliminan esa restricción y abren la posibilidad de explotar paralelismo de memoria compartida directamente en el intérprete.

OMP4Py es una biblioteca que adapta el modelo de programación OpenMP al entorno Python [3, 4]. Permite expresar paralelismo mediante directivas integradas en el código, con una sintaxis próxima a la de OpenMP en C o C++ [5]. La biblioteca ofrece cuatro modos de ejecución (desde un intérprete puro hasta kernels Cython con tipos nativos), lo que permite comparar el coste de cada capa de abstracción sobre el mismo código fuente.

Este TFM evalúa OMP4Py sobre NAS Parallel Benchmarks adaptados a Python. Los NAS son adecuados para este propósito porque definen problemas conocidos, clases de tamaño estandarizadas y rutinas de verificación propias [6]. El trabajo no consiste en validar implementaciones triviales. Se trata de portar benchmarks completos, conservar su semántica y medir si OMP4Py puede explotarlos de forma paralela y eficiente en un entorno de supercomputación real.

1.1. Motivación

La pregunta central del TFM es si un programador puede expresar paralelismo de estilo OpenMP en Python con OMP4Py, obtener resultados correctos y conseguir aceleración real sobre más de un hilo.

La motivación surge de una tensión habitual en computación científica de altas prestaciones [7]. Python es cómodo para desarrollar, pero los kernels numéricos intensivos suelen requerir código nativo o bibliotecas como NumPy [8], que delegan el trabajo paralelo a capas C sin exponer el modelo de programación al desarrollador. OMP4Py propone una vía intermedia. Mantiene una interfaz Python y, cuando el rendimiento lo exige, compila los kernels mediante Cython. Este TFM estudia esa vía con benchmarks estandarizados, separando el efecto del modo de ejecución del efecto del paralelismo entre hilos.

Durante la evaluación, los dos factores no pesaron por igual. El tipado Cython resultó decisivo, mucho más que el número de hilos, y la escalabilidad entre hilos dependió sobre todo de la estructura de cada kernel. Las versiones de Python *free-threaded* evaluadas se comportaron de forma parecida una vez corregida la afinidad del entorno. Las cifras concretas se presentan en los Capítulos 5 y 7.

La motivación se concreta en los objetivos de la sección siguiente y en las contribuciones resumidas en la Sección 1.3.

1.2. Objetivos

El objetivo general es desarrollar y evaluar aplicaciones paralelas con OMP4Py usando NAS Parallel Benchmarks como conjunto de referencia en el cluster Finisterrae III.

Los objetivos específicos se recogen en la Tabla 1.1, identificados como O1–O7 para poder referenciarlos a lo largo de la memoria.

Tabla 1.1: Objetivos específicos del trabajo.

ID	Objetivo específico
O1	Estudiar el modelo de programación de OMP4Py y sus cuatro modos de ejecución.
O2	Adaptar cinco NAS Parallel Benchmarks a OMP4Py conservando la verificación oficial de cada uno como criterio de aceptación.
O3	Construir una infraestructura reproducible para ejecutar los benchmarks en Finisterrae III mediante Slurm.
O4	Medir rendimiento y escalabilidad con los cuatro modos de ejecución, distintas clases de problema y entre 1 y 32 hilos.
O5	Comparar el impacto del tipado Cython frente al aumento del número de hilos en cada benchmark.
O6	Comparar el comportamiento de Python 3.13t, Python 3.14t y Python 3.15t en modo <i>free-threaded</i> .
O7	Identificar mediante profiling los cuellos de botella en los casos donde la escalabilidad sea reducida, usando perf stat y el nuevo sistema de profiling de Python 3.15 (muestreo y trazado) para medir el coste de sincronización, el recuento de operaciones del runtime y el balance de carga.

1.3. Contribuciones

La memoria aporta cuatro contribuciones principales.

1. Un port verificado de CG, EP, FT, IS y MG a OMP4Py, manteniendo la interfaz común de ejecución, las clases NAS y las rutinas internas de verificación.
2. Una infraestructura reproducible de campañas en Finisterrae III basada en Slurm, con generación de matrices experimentales, registros por ejecución y consolidación automática de resultados.
3. Una caracterización experimental de OMP4Py con Python 3.15t en los cuatro modos de ejecución cuando el coste lo permite, separando el efecto del tipado Cython, el tamaño de clase y el número de hilos.
4. Una comparación entre Python 3.13t, 3.14t y 3.15t, junto con un diagnóstico de escalabilidad mediante perf stat y el nuevo sistema de profiling de Python 3.15.

El código de los benchmarks adaptados y de la infraestructura de campañas está disponible en el repositorio público https://github.com/alexvedo/omp4py_TFM.

Estas contribuciones no se plantean como una evaluación genérica de Python *free-threaded*. El objeto de estudio es OMP4Py sobre aplicaciones NAS completas, con especial atención a distinguir entre corrección funcional, mejora por compilación y escalabilidad paralela efectiva.

1.4. Alcance

El trabajo cubre los benchmarks EP, CG, IS, MG y FT. Son los cinco sobre los que se ha introducido paralelismo explícito con directivas OMP4Py y cuya verificación es correcta en todos los modos. Los benchmarks BT, SP y LU quedan fuera del alcance principal. Sus estructuras de datos y patrones de sincronización presentan restricciones que exceden el tiempo disponible para este TFM, y sus adaptaciones actuales no incorporan paralelismo.

Las modificaciones a la propia biblioteca OMP4Py también quedan fuera del alcance. La biblioteca se toma como entorno dado. El trabajo se centra en las aplicaciones y en su evaluación experimental.

La campaña experimental principal cubre las clases de problema S, W y A. La clase S permite comparar los cuatro modos de ejecución y validar la corrección en todos los benchmarks. La clase W se centra en los modos compilados y la clase A en el modo 3, que es el único modo práctico para cargas grandes. Además, se ejecuta una campaña de comparación entre

Python 3.13t, 3.14t y 3.15t sobre las clases S y W. Todas las ejecuciones finales obtuvieron verificación correcta.

1.5. Estructura de la memoria

El Capítulo 2 describe el contexto tecnológico, incluyendo Python *free-threaded*, OMP4Py y los NAS Parallel Benchmarks. El Capítulo 3 detalla el proceso de adaptación de los cinco benchmarks y las decisiones de implementación más relevantes. El Capítulo 4 define la metodología experimental y la infraestructura de medición. El Capítulo 5 presenta y analiza los resultados de la campaña principal con Python 3.15t. El Capítulo 6 diagnostica con *perf stat* la ocupación real de CPU, el IPC y los límites de escalabilidad, y con el nuevo sistema de profiling de Python 3.15 (muestreo y trazado) los costes de sincronización, el recuento de operaciones del runtime y el balance de carga. El Capítulo 7 compara Python 3.13t, 3.14t y 3.15t. El Capítulo 8 resume las principales conclusiones y plantea el trabajo futuro.

Contexto tecnológico

2.1. Python y el problema del GIL

Python gestiona la memoria mediante contadores de referencia. Para evitar condiciones de carrera en esos contadores cuando múltiples hilos acceden al mismo objeto, el intérprete CPython mantiene el GIL, un mutex global que solo permite a un hilo ejecutar bytecode Python en cada instante. La consecuencia práctica es que los programas Python multihilo no pueden aprovechar múltiples núcleos para trabajo intensivo en el intérprete, incluso en sistemas con decenas de cores disponibles.

La restricción no impide toda forma de paralelismo, porque Python puede delegar cómputo a extensiones nativas o distribuir trabajo entre procesos. Sin embargo, esas alternativas cambian el modelo de programación. El punto relevante para este trabajo es el paralelismo de memoria compartida expresado directamente en código Python, con hilos y directivas de estilo OpenMP. La relación con otros enfoques se resume en la Sección 2.4.

Las versiones *free-threaded* de Python eliminan el GIL. Disponibles de forma experimental desde Python 3.13t [2], sustituyen los contadores de referencia por mecanismos atómicos y biased locking. Esto añade un pequeño sobrecoste en código monohilo, pero hace posible la ejecución paralela real con hilos Python. Este TFM utiliza Python 3.15.0b1t como entorno principal y compara sus resultados con Python 3.13.13t y Python 3.14.4t, compilados también con el GIL desactivado.

2.2. OMP4Py

OMP4Py es una implementación del modelo de programación OpenMP para Python [3, 4]. Su objetivo es permitir que el programador exprese regiones paralelas, distribución de iteraciones y sincronización mediante directivas integradas en código Python, con una sintaxis análoga a la de OpenMP en C, C++ o Fortran [5, 9].

Las directivas se expresan como gestores de contexto.

```
with omp("parallel for"):
    for i in range(n):
        ...
```

La biblioteca ofrece cuatro modos de ejecución, seleccionables en tiempo de ejecución mediante el argumento `-m`.

Modo	Nombre	Descripción
0	puro	Runtime puro de OMP4Py en Python, kernels sin compilar.
1	híbrido	Runtime nativo (Cython) si está disponible, kernels sin compilar.
2	compilado	Kernels compilados a Cython, tipos inferidos.
3	compilado tipado	Cython con anotaciones de tipo explícitas y memoryviews.

En los cuatro modos, las directivas se transforman mediante el árbol sintáctico abstracto (AST) del módulo `ast` de Python y el paralelismo se ejecuta con hilos reales de la API `threading`, de modo que en Python *free-threaded* pueden aprovechar múltiples núcleos. La diferencia entre modos está en qué partes del código se compilan (Figura 2.1). En los modos 0 y 1 los kernels del usuario no se compilan. El modo 0 fuerza el runtime puro de OMP4Py, cuyas primitivas de distribución de trabajo y sincronización están implementadas en Python, y es la línea base funcional. El modo 1 usa el runtime por defecto, que emplea la implementación nativa del propio runtime, compilada con Cython, cuando está disponible. Los modos 2 y 3 compilan además los kernels del usuario a C mediante Cython [10]. La diferencia entre ellos es que el modo 3 permite anotar variables con tipos Cython (`cython.double`, `cython.int`, `memoryviews`) para eliminar la indirección de los objetos Python en los bucles numéricos.

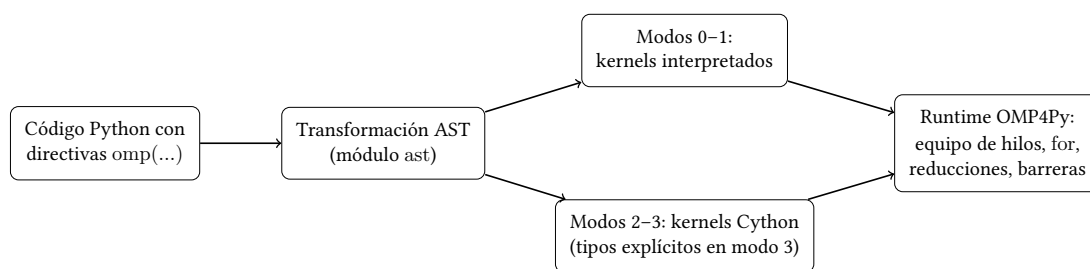


Figura 2.1: Flujo de ejecución de un programa OMP4Py. Las directivas se procesan mediante transformación AST en todos los modos. El modo determina si los kernels del usuario se compilan con Cython y qué implementación del runtime ejecuta la distribución de trabajo y la sincronización.

La compilación en los modos 2 y 3 se realiza en tiempo de ejecución, antes del primer uso de cada kernel. Los kernels compilados se almacenan en caché para reutilizarlos en ejecuciones posteriores.

Esta diversidad de modos permite aislar tres factores de rendimiento, el coste del runtime Python frente al runtime compilado, el impacto del tipado Cython sobre los bucles numéricos y la ganancia real del paralelismo entre hilos.

2.3. NAS Parallel Benchmarks

Los NAS Parallel Benchmarks (NPB) son un conjunto de benchmarks desarrollado por NASA para evaluar sistemas paralelos [6]. Cada benchmark implementa un núcleo computacional representativo de aplicaciones de simulación numérica.

- **EP** (*Embarrassingly Parallel*), generación de pares aleatorios gaussianos mediante el método de Box-Muller. El trabajo útil es embarazosamente paralelo. La única sincronización es una reducción global al final.
- **CG** (*Conjugate Gradient*), estimación del menor autovalor de una matriz dispersa irregular mediante gradiente conjugado. Requiere productos matriz-vector, actualizaciones de vectores y reducciones escalares en cada iteración.
- **IS** (*Integer Sort*), ordenación entera mediante un algoritmo de counting sort por cubos. Exige paralelismo en histogramas, offsets y ordenación local con semántica estricta.
- **MG** (*Multi-Grid*), resolución aproximada de la ecuación de Poisson 3D mediante el método multigrid. Opera sobre mallas en varios niveles de resolución con operaciones de restricción, prolongación e interpolación.
- **FT** (*Fourier Transform*), solución de una EDP mediante transformadas de Fourier 3D complejas. El kernel principal es una FFT aplicada sucesivamente a cada dimensión de un array tridimensional de números complejos.

Cada benchmark define varias clases de problema (S, W, A, B, C, D), asociadas a tamaños de malla o números de iteraciones crecientes. Esta escala permite estudiar el comportamiento de OMP4Py en cargas de distinta granularidad sin cambiar de benchmark.

Los NPB incluyen rutinas de verificación internas que comprueban la exactitud numérica del resultado. Esa verificación es la única condición de aceptación utilizada en este trabajo. Una adaptación que ejecute más rápido pero no verifique correctamente se rechaza.

2.4. Trabajo relacionado

El paralelismo en Python se ha abordado tradicionalmente por vías que evitan el GIL en lugar de eliminarlo. Algunas delegan el cómputo a bibliotecas nativas, otras compilan sub-

conjuntos del lenguaje y otras distribuyen el trabajo entre procesos. La Tabla 2.1 resume la posición de OMP4Py frente a esas alternativas.

Enfoque	Modelo de programación	Relación con este TFM
NumPy [8]	Operaciones vectoriales ejecutadas en código nativo, habitualmente con liberación del GIL.	Muy eficiente cuando el algoritmo encaja en primitivas vectoriales, pero el paralelismo no se expresa como regiones, bucles y sincronización de memoria compartida al nivel del programa Python.
Cython [10] y Numba [11]	Compilación de funciones numéricas a código nativo, con restricciones diferentes sobre tipos y constructos soportados.	Reducen el coste del intérprete. En OMP4Py, Cython es precisamente la capa que permite estudiar cuánto aporta el tipado frente al paralelismo entre hilos.
PyOMP [12]	Directivas OpenMP sobre funciones compiladas con Numba.	Es la alternativa más próxima por usar directivas, pero queda ligada al subconjunto compilable por Numba. OMP4Py parte de código Python con directivas propias y compila opcionalmente los kernels mediante Cython.
multiprocessing, mpi4py [13] y Dask [14]	Paralelismo mediante procesos y, en mpi4py/Dask, posible distribución entre nodos.	Escalan más allá de un proceso, pero cambian el modelo a memoria privada y comunicación explícita o gestionada. Este TFM se centra en memoria compartida y equipos de hilos al estilo OpenMP.
OMP4Py [3, 4]	Directivas de estilo OpenMP integradas en código Python, con runtime de hilos y modos interpretados o compilados.	Permite evaluar, sobre el mismo código fuente, el coste del runtime Python, el efecto del tipado Cython y la escalabilidad real en Python <i>free-threaded</i> .

Tabla 2.1: Relación entre OMP4Py y otros enfoques de paralelismo o aceleración en Python.

La eliminación del GIL en las versiones *free-threaded* [2] abre una vía distinta, la de ejecutar hilos Python reales sobre memoria compartida. OMP4Py se sitúa en esa vía al trasladar el modelo de directivas de OpenMP [5, 9] al código Python. A diferencia de los trabajos que presentan la propia biblioteca, esta memoria evalúa su comportamiento sobre aplicaciones completas, con verificación NAS, cuatro modos de ejecución y varias versiones del intérprete.

Como cargas de trabajo se emplean los NAS Parallel Benchmarks [6], ampliamente utilizados para evaluar sistemas paralelos. Existen portados previos de los NPB a Python a partir de las versiones secuenciales en C++ [15], que sirven de base secuencial para la adaptación descrita en el Capítulo 3. Hasta donde alcanza este trabajo, la evaluación de OMP4Py sobre

kernels NPB completos, con comparación sistemática de modos y de versiones *free-threaded*, no se había abordado.

Adaptación de los NAS Parallel Benchmarks a OMP4Py

3.1. Benchmarks adaptados

Se adaptaron cinco NAS Parallel Benchmarks al modelo de programación OMP4Py, en concreto EP, CG, IS, MG y FT. En todos los casos se conservaron la clase del problema, el número de iteraciones y la verificación oficial del benchmark como criterio único de aceptación. Los tres benchmarks restantes del conjunto NPB (BT, SP y LU) quedan diferidos por dos motivos distintos. BT y SP se apoyan en el método ADI, con resoluciones tridiagonales a lo largo de cada eje. Como las líneas son independientes entre sí, su paralelización encaja en un `parallel for` ordinario, y quedaron fuera sobre todo por el tiempo disponible. El caso de LU es distinto. Su barrido SSOR (rutinas `blts` y `buts`) arrastra una dependencia en *wavefront*, porque cada punto necesita a sus vecinos ya actualizados en las tres direcciones, así que no admite un bucle paralelo directo. Las formulaciones habituales en OpenMP lo resuelven de tres maneras, con el constructo `ordered` (bucles *doacross*), con sincronización punto a punto mediante `flush` y `atomic`, o con tareas y dependencias (`depend`). El conjunto de directivas que expone OMP4Py en su versión actual no incluye ninguno de esos mecanismos, así que un port directo del LU paralelo de NPB no llega a ser expresable. Solo cabría reescribirlo por hiperplanos apoyándose en `parallel for` y `barrier`, que supone un trabajo algorítmico aparte. En las adaptaciones actuales, los tres benchmarks se mantienen en versión serial, con compilación Cython pero sin directivas OMP4Py.

Todos los benchmarks siguen una misma interfaz de línea de comandos, basada en las opciones `-c` para la clase NAS, `-t` para el número de hilos y `-m` para el modo de ejecución. La invocación completa se recoge en el Anexo A.

La Tabla 3.1 resume el estado funcional de los ocho benchmarks disponibles en el repositorio.

Benchmark	Estado	Observación principal
EP	Completo	Reducción paralela y variables privadas tipadas
CG	Completo	Producto disperso y reducciones escalares
IS	Completo	Semántica de cubos revisada para OMP4Py
MG	Completo	Kernels de malla con memoryviews 1D
FT	Completo	FFT compleja con región paralela exterior
BT	Diferido	Sin directivas OMP4Py, solo compilación Cython
SP	Diferido	Sin directivas OMP4Py, solo compilación Cython
LU	Diferido	Sin directivas OMP4Py, solo compilación Cython

Tabla 3.1: Estado de los benchmarks NAS en el repositorio.

3.2. Metodología de adaptación

El port se realizó de forma incremental, mediante un ciclo de portar, ejecutar, verificar y depurar. Se partió de las versiones Python seriales de referencia [15], derivadas de los NAS en C++, y cuando existía se consultó la versión C/OpenMP original para entender el patrón de paralelismo. Los pasos seguidos fueron los siguientes.

1. Partir de la versión Python serial NAS.
2. Comparar con la versión C/OpenMP cuando existe.
3. Conservar clase, tamaño, iteraciones, salida y verificación NAS.
4. Eliminar dependencias de Numba [11] y adaptar imports.
5. Añadir la interfaz común -c, -t, -m.
6. Conseguir verificación correcta en modo 0 antes de cualquier otra modificación.
7. Separar los kernels numéricos del código de orquestación (temporizadores, E/S, verificación).
8. Compilar solo los kernels y pasar tamaños y constantes como argumentos.
9. Cambiar el indexado NumPy encadenado (`u[k][j][i]`) por indexado directo (`u[k, j, i]`).
10. Añadir anotaciones de tipo Cython y memoryviews en el modo 3.

La verificación oficial del benchmark fue el criterio de aceptación. Una optimización solo se mantenía si seguía dando SUCCESSFUL, y las versiones más rápidas que no verificaban se descartaban. Las decisiones de implementación que este criterio exigió se detallan en la Sección 3.3.

3.3. Decisiones de implementación

3.3.1. Separación de kernels y orquestación

Las funciones de alto nivel de los NAS mezclan inicialización, temporizadores, E/S, selección de clase, verificación y kernels numéricos. Compilar esas funciones completas con OMP4Py resulta frágil porque la transformación AST no puede aplicarse de forma segura a código que combina efectos secundarios con regiones paralelas. Por ello, la orquestación se mantuvo en Python puro y solo se compilaban las funciones con carga numérica significativa.

3.3.2. Compilación dinámica tras inicialización

En varios benchmarks, las constantes y tamaños de array dependen de la clase seleccionada y se calculan durante la ejecución. Si un kernel compilado captura esos valores antes de que la inicialización se complete, puede usar dimensiones incorrectas. Para evitarlo, la decoración OMP4Py se aplica después de completar la inicialización de constantes, en una función `compile_kernels()` que se invoca explícitamente una vez conocida la clase.

3.3.3. Indexado directo en NumPy

Las expresiones como `u[k][j][i][m]` generan objetos intermedios en NumPy y dificultan la generación de código eficiente por parte de Cython. La sustitución por `u[k, j, i, m]` es semánticamente equivalente y permite a Cython generar accesos directos a los buffers C cuando se combina con `memoryviews`.

3.3.4. Variables privadas explícitas

En EP apareció un fallo de verificación en modo compilado tipado. Varias variables temporales se compartían entre hilos y producían resultados incorrectos. La corrección consistió en declararlas como privadas dentro de la región paralela. Este caso ilustra que una traducción estructuralmente parecida al código C/OpenMP no garantiza una semántica equivalente en Python. La visibilidad de las variables locales depende del ámbito léxico del intérprete, no de las reglas de ámbito de OpenMP.

3.3.5. Funciones auxiliares compilables localmente

Antes de la campaña final se corrigieron varios cuellos de botella derivados de funciones auxiliares definidas fuera del propio fichero del benchmark. En EP, la función `vranlc`, encargada de generar números pseudoaleatorios, se llamaba como función Python externa desde el

kernel compilado, lo que impedía que el modo 2 compilase todo el cuerpo relevante. La solución fue incluir en `EP_Python.py` versiones localmente compilables de `vranlc`, una sin tipos y otra tipada. En IS se revisó la inicialización de la secuencia de claves para que cada modo ejecutase la ruta correcta y no quedase una parte secuencial accidental en modo híbrido. Junto con la eliminación del estado global en MG que se describe más adelante, estas correcciones separan los límites del port de los límites del runtime. En las versiones finales, los kernels críticos están dentro del propio fichero del benchmark y son compilables por OMP4Py/Cython.

3.4. Particularidades por benchmark

3.4.1. EP

EP genera pares de variables aleatorias con distribución normal mediante el método de Box-Muller. El kernel paralelo central recorre 2^{nk} bloques de dos muestras cada uno. Dentro de cada bloque calcula una semilla local, evalúa raíces cuadradas y aritmética de punto flotante en escalares, y acumula dos contadores globales (`sx`, `sy`) con una reducción.

La implementación tiene dos versiones compiladas, `ep_compute` (modo 2) y `ep_compute_types` (modo 3). Ambas usan la misma directiva `parallel reduction(+:sx,sy)` con planificación estática, pero en `ep_compute_types` todas las variables del bucle interno están declaradas con tipos C de Cython (`cython.int`, `cython.double`) y se declaran explícitamente privadas para evitar condiciones de carrera. El módulo principal selecciona la versión tipada en modo 3 mediante `use_compiled_types()`. Sin ese tipado, cada operación aritmética pasa por el mecanismo de despacho de objetos Python. La magnitud de esta diferencia se cuantifica en el Capítulo 5.

La sección crítica al final de la región paralela actualiza el histograma de pares. Esta operación es secuencial y ocupa un porcentaje despreciable del tiempo total en modo 3.

3.4.2. CG

CG resuelve un sistema lineal disperso con el método del gradiente conjugado y calcula el autovalor más pequeño de la matriz. El bucle de gradiente conjugado tiene 25 iteraciones. Cada una realiza un producto matriz-vector disperso seguido de varias operaciones de reducción.

La implementación distingue entre modo no compilado y modo compilado. En los modos 0 y 1, la función `conj_grad` usa directivas `parallel for` independientes para cada bucle. En los modos 2 y 3, la función `conj_grad_compiled` envuelve todo el bucle de iteraciones en una única directiva `parallel`, de modo que el grupo de hilos se crea una sola vez por llamada y los bucles internos se distribuyen con `for`. Esta distinción reduce el overhead de creación y

destrucción de hilos, que es especialmente relevante en clase S donde los tiempos de kernel son del orden de décimas de segundo.

Los kernels auxiliares (`normalize_z_compiled`, `shift_column_indices_compiled`, `fill_vector_compiled`) están anotados con `memoryviews` de Cython (`cython.long[:]`, `cython.double[:]`) para el acceso a arrays. La versión tipada evita que las operaciones escalares de los bucles internos pasen por objetos Python. El efecto de esta decisión sobre el rendimiento absoluto se cuantifica en el Capítulo 5.

Desde el punto de vista del port, el rasgo más importante de CG es que cada iteración combina trabajo disperso con varias reducciones escalares. El impacto de esa estructura sobre la escalabilidad se analiza en los Capítulos 5 y 6.

3.4.3. IS

IS ordena enteros de 32 bits mediante un bucket sort en dos fases. La primera construye el histograma de claves y la segunda realiza la ordenación local en cada cubo. Para inicializar el vector de claves, el módulo principal usa `create_seq` (función Python sin compilar, modos 0/1/2) o `create_seq_types` (compilada con Cython y tipos explícitos, modo 3). Además, las rutas críticas de ordenación usan variables privadas y tipos Cython explícitos en modo 3. El efecto de esta distinción se cuantifica en el Capítulo 5.

La función `rank` usa una única directiva `parallel` con un bloque de variables privadas explícitas por hilo. Cada hilo gestiona su identificador, los offsets de cubos, las posiciones de inserción y los contadores locales. Una directiva `barrier` explícita separa la fase de histograma de la fase de ordenación. Todos los hilos deben completar el conteo global de claves antes de calcular los offsets acumulados, que se calculan en una sección secuencial.

El paso de prefijos acumulados queda como una fase secuencial entre las dos fases paralelas. En este capítulo interesa como restricción de implementación, y su peso en tiempo y escalabilidad se estudia después.

3.4.4. MG

MG aplica un ciclo en V del método multigrid para resolver una ecuación de Poisson tridimensional discreta. Los kernels del ciclo son `psinv` (suavizado), `resid` (cálculo de residuo), `rprj3` (restricción), `interp` (interpolación), `comm3` (comunicación de fronteras) y `zero3` (inicialización).

Durante el port se eliminaron dependencias de estado global en kernels como `zran3` y `norm2u3`, y se añadieron rutas tipadas para que Cython pudiese compilar los accesos a malla y los acumuladores escalares sin atravesar el intérprete. La diferencia entre modo 2 y modo 3 es muy grande en tiempo absoluto, como recoge el Capítulo 5.

Los arrays de malla se representan como vectores 1D con indexado manual (`r[(i3*n2 + i2)*n1 + i1]`) porque las `memoryviews` de Cython en sus versiones más accesibles solo funcionan eficientemente con arrays de forma fija o con vistas 1D. El bucle más externo de cada kernel (`for i3 in range`) se distribuye entre hilos con `with omp("for")`.

La particularidad de MG es la jerarquía de niveles. El ciclo en V desciende hasta una malla de $2 \times 2 \times 2$. En los niveles gruesos, el bucle externo tiene solo unas pocas iteraciones (1 o 2), lo que hace imposible distribuir trabajo entre muchos hilos. El efecto de esa granularidad desigual se cuantifica en el Capítulo 6.

3.4.5. FT

FT calcula la transformada de Fourier tridimensional discreta de un array complejo mediante FFTs 1D sucesivas sobre cada eje (tres pasadas). La función principal abre una única directiva `parallel` que agrupa todos los hilos para el cómputo completo. Dentro de esa región, cada pasada llama a `ffts1/2/3`, que distribuye las filas del array entre hilos con `for` y delega el cómputo a `cfftz` y `fftz2`. El patrón de región `parallel` exterior persistente evita crear y destruir el grupo de hilos en cada pasada de la FFT.

La implementación tiene una jerarquía completa de funciones tipadas, `fft_types`, `ffts1/2/3_types`, `cfftz_types`, `fftz2_types`. En sus versiones no tipadas, las operaciones sobre pares complejos usan el tipo `complex` de Python, que almacena parte real e imaginaria como objetos separados y llama a funciones Python para la suma y el producto. En las versiones tipadas, esos valores son `cython.doublecomplex` (`double complex` de C99), y las operaciones aritméticas son instrucciones de punto flotante del procesador. Esta diferencia es especialmente visible porque cada iteración del bucle interior de `fftz2` realiza dos multiplicaciones y dos sumas sobre complejos. El impacto se reporta en los capítulos de resultados.

La pasada sobre el eje x debe completarse antes que la del eje y , y esta antes que la del eje z . Dentro de cada pasada las filas son independientes y el `parallel for` funciona, pero la separación entre pasadas introduce barreras naturales que se analizan en el Capítulo 6.

Metodología experimental

4.1. Objetivo de la evaluación

La evaluación experimental mide tres aspectos del comportamiento de OMP4Py sobre los cinco benchmarks adaptados. Son la corrección, el rendimiento absoluto y la escalabilidad. La corrección se comprueba mediante la verificación oficial de cada benchmark. El rendimiento se mide con el tiempo interno reportado por NAS y con los Mop/s de salida. La escalabilidad se estudia comparando el mejor tiempo observado con 1, 2, 4, 8, 16 y 32 hilos dentro del mismo modo, clase e intérprete. Como control de robustez, también se calcula la mediana y el rango de las repeticiones válidas.

La comparación entre modos de ejecución articula el estudio. Los modos 0 y 1 sirven como referencia funcional y permiten cuantificar el coste de interpretar directivas OMP4Py sin compilación. Los modos 2 y 3 son las configuraciones orientadas a rendimiento, y la diferencia entre ambos mide el impacto del tipado Cython sobre los kernels numéricos.

4.2. Entorno de ejecución

Los experimentos se ejecutan en Finisterrae III, el supercomputador del Centro de Supercomputación de Galicia (CESGA), mediante el gestor de trabajos Slurm. Se utilizan nodos de tipo clk, con procesadores Intel Xeon de 48 núcleos y memoria DDR4. Las ejecuciones se limitan a 32 hilos para facilitar la planificación de Slurm y cubrir los rangos de baja, media y alta concurrencia dentro de un nodo.

El intérprete principal es Python 3.15.0b1t, compilado con soporte *free-threaded* (`-disable-gil`) en el directorio de usuario. Para la comparación entre versiones se utilizaron también Python 3.13.13t y Python 3.14.4t, ambos compilados como intérpretes *free-threaded*. En las tres versiones de la campaña comparativa se comprobó que el GIL estaba desactivado.

Para aligerar tablas, figuras y análisis, el resto del documento usa las formas abreviadas Python 3.13t, Python 3.14t y Python 3.15t. Estas abreviaturas se refieren, respectivamente, a las compilaciones exactas Python 3.13.13t, Python 3.14.4t y Python 3.15.0b1t descritas aquí.

El entorno de ejecución se automatiza mediante `scripts/ft3/setup_venvs.sh`, que crea un entorno virtual por versión de Python e instala las dependencias necesarias. Las campañas finales usaron NumPy 2.4.6 y Cython 3.2.5 en los tres intérpretes. El código fuente se mantuvo idéntico entre versiones para que la variable experimental fuese el intérprete y no la implementación del benchmark.

Las campañas finales no fijan por defecto variables de afinidad de OpenMP como `OMP_PROC_BIND`, `OMP_PLACES` o `GOMP_CPU_AFFINITY`. Estas variables se eliminan explícitamente en los scripts de ejecución salvo que se active un modo experimental específico. La decisión evita que la política de afinidad del entorno distorsione la comparación entre versiones y recuentos de hilos.

4.3. Automatización de la campaña

La ejecución en Finisterrae III se automatiza con los scripts de `scripts/ft3`. El script `make_matrix.py` genera la matriz de combinaciones de cada campaña, `submit_campaign.sh` crea el directorio de resultados y lanza el array de Slurm, `array_run.sbatch` ejecuta los trabajos, `run_one.py` ejecuta una combinación concreta y `parse_results.py` reconstruye las tablas `summary.csv`, `summary_best.csv` y los registros JSON. La Figura 4.1 resume este flujo.

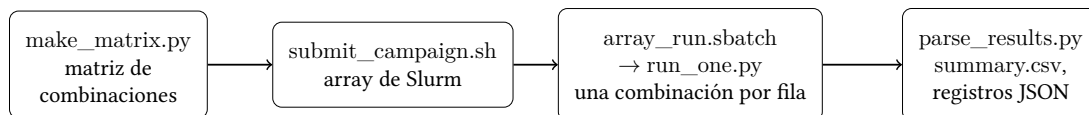


Figura 4.1: Flujo de la infraestructura de campaña en Finisterrae III. Generación de la matriz de combinaciones, envío como array de Slurm, ejecución de cada combinación y consolidación de resultados.

Cada fila de la matriz invoca el benchmark correspondiente mediante la interfaz común descrita en el Capítulo 3 (opciones `-c`, `-t` y `-m`). Para cada ejecución se registra la salida estándar, la salida de error, el código de retorno, el tiempo de proceso externo, el tiempo interno NAS, los Mop/s y el estado de verificación. Las combinaciones se distribuyen entre múltiples trabajos Slurm mediante arrays (`-array`), donde cada elemento ejecuta varias filas de la matriz de forma secuencial. Esta organización evita una ejecución monolítica, limita el número de trabajos enviados al planificador y produce ficheros con nombres semánticos, no dependientes del identificador interno de Slurm.

Una ejecución se considera válida si el proceso termina con código de retorno 0 y la verificación reporta SUCCESSFUL. Las ejecuciones con error, timeout o verificación incorrecta se conservan en los resultados como indicadores de limitaciones del modo o la implementación.

4.4. Diseño de la matriz experimental

La evaluación se divide en dos bloques, una campaña principal con Python 3.15.0b1t y una campaña comparativa entre versiones *free-threaded*.

La campaña principal se divide en tres bloques por clase, y una campaña comparativa añade la variación de intérprete. La Tabla 4.1 resume las cuatro configuraciones. Todas ejecutan los cinco benchmarks (CG, EP, FT, IS, MG) con 1, 2, 4, 8, 16 y 32 hilos. Difieren en el intérprete, la clase, los modos evaluados y el número de repeticiones.

4.4.1. Campaña principal en clase S

La clase S es la más pequeña de las NAS y permite comparar los cuatro modos en un tiempo razonable. Su papel principal es validar la corrección en todas las combinaciones y cuantificar el coste relativo de cada modo con un único hilo.

4.4.2. Campaña principal en clase W

La clase W es una carga de tamaño intermedio. Se restringe a los modos 2 y 3 porque los modos 0 y 1 producen tiempos excesivamente altos en cargas de este tamaño y no aportan información adicional sobre paralelismo real.

4.4.3. Campaña principal en clase A

La clase A es la carga principal para analizar escalabilidad. En problemas de mayor tamaño, el cociente entre trabajo útil y overhead de paralelización es más favorable, por lo que esta campaña resulta más informativa. Se ejecuta únicamente el modo 3 porque las clases S y W ya muestran que es el único modo práctico para cargas grandes.

4.4.4. Campaña comparativa entre versiones

La campaña comparativa mantiene el mismo código fuente y ejecuta Python 3.13.13t, Python 3.14.4t y Python 3.15.0b1t. Cubre las clases S y W. Los modos compilados 2 y 3 se ejecutan en ambas clases, y los modos interpretados 0 y 1 se añaden en clase S, donde el código no compilado todavía termina en un tiempo razonable y permite observar si la versión del intérprete influye en la ejecución no compilada. La clase S usa cinco repeticiones por combinación y la clase W usa tres.

Parámetro	Campana principal			Comparativa versiones
	Clase S	Clase W	Clase A	
Python	3.15.0b1t	3.15.0b1t	3.15.0b1t	3.13.13t, 3.14.4t, 3.15.0b1t
Clase	S	W	A	S, W
Modos	0, 1, 2, 3	2, 3	3	S (0-3), W (2, 3)
Repeticiones	3	3	3	S (5), W (3)

Tabla 4.1: Matriz experimental. Todas las configuraciones ejecutan CG, EP, FT, IS y MG con 1, 2, 4, 8, 16 y 32 hilos.

4.4.5. Campana de profiling

Se lanzó una campana específica de profiling con perf stat [16] para relacionar los tiempos NAS con ocupación de CPU, IPC y eventos de memoria. Esta campana no repite toda la matriz experimental, sino que se concentra en los casos necesarios para explicar por qué EP y FT escalan y por qué CG, IS y MG no obtienen aceleración sostenida. Para evitar mezclar efectos de versión con el diagnóstico de bajo nivel, el cuerpo de la memoria usa una matriz homogénea, con los cinco benchmarks en clase W, modo 3, con Python 3.15t y los recuentos de 1, 8 y 32 hilos. Los óptimos finos y la comparación entre versiones se analizan en sus campañas correspondientes, y perf stat se usa aquí como apoyo diagnóstico sobre la configuración principal.

Parámetro	Valores
Herramienta	perf stat
Benchmarks	CG, EP, FT, IS y MG
Python	3.15.0b1t
Clase y modo	W, modo 3
Hilos	1, 8 y 32
Métricas	CPUs utilizadas, ciclos, instrucciones, IPC y eventos de caché

Cada fila de profiling ejecuta primero una repetición de calentamiento y después una repetición medida con perf stat. Esta separación reduce el impacto de la compilación Cython en las métricas de rendimiento. Aun así, las métricas de perf stat corresponden al proceso completo, no solo a la ventana temporizada por NAS. En benchmarks muy cortos, como IS clase W, esta limitación obliga a interpretar los contadores como apoyo diagnóstico y no como medida exacta del kernel.

4.5. Justificación de la restricción de modos en clases grandes

Los modos 0 y 1 no incluyen compilación Cython. En clase S, el modo 0 ya multiplica los tiempos del modo 3 por factores que van de $16\times$ en FT a $381\times$ en MG. Extender esos modos

a clases W o A consumiría recursos de cluster desproporcionados sin añadir información relevante sobre la escalabilidad paralela, que es el objetivo de las clases mayores. Por eso los modos 0 y 1 se evalúan por completo solo en clase S.

La misma lógica justifica no ejecutar modo 2 en clase A dentro de la campaña principal. El modo 2 es útil para cuantificar el efecto del tipado en clases S y W, pero sus tiempos absolutos en W ya muestran que no es una configuración práctica para cargas grandes. La comparación entre versiones se restringe a S y W por el mismo motivo, ya que permite estudiar el efecto del intérprete sin multiplicar el coste de la campaña con clase A.

4.6. Métricas y tratamiento de los resultados

La métrica principal es el tiempo NAS interno, es decir, el tiempo que el propio benchmark mide entre el inicio y el fin del cómputo, excluyendo inicialización, E/S y verificación. Los Mop/s reportados por el benchmark se utilizan como métrica complementaria.

El análisis principal se basa en el mejor tiempo de las repeticiones válidas para cada combinación. Esta elección es habitual en benchmarking de rendimiento, porque aproxima el tiempo de ejecución menos perturbado por ruido del sistema y evita que una interrupción puntual del nodo domine la comparación. Para que esa decisión no dependa de una sola ejecución favorable, la campaña conserva todas las repeticiones y calcula también mediana, coeficiente de variación y rango.

La Tabla 4.2 resume esa estabilidad. En la campaña principal, el coeficiente de variación mediano se mantiene entre 0.59 % y 0.75 % según la clase. En los modos compilados de la comparación entre versiones queda en 0.61–0.64 %, y allí la diferencia mediana entre la mediana y el mejor tiempo es inferior al 0.5 %. Los modos interpretados de esa comparación son algo más variables, con un coeficiente de variación mediano del 1.59 % en clase S, debido sobre todo a EP y FT en ejecuciones largas no compiladas, aunque la mediana se mantiene cerca del mejor tiempo. Por tanto, las conclusiones no dependen de escoger una repetición aislada. La mediana conduce a la misma lectura, pero el mejor tiempo se mantiene como referencia para calcular speedups y comparar rendimiento.

Campaña	Clase	Comb.	Reps.	CV med.	CV p95	Rango med.	Mediana/mejor med.
Principal	S	117	351	0.67 %	5.55 %	1.52 %	0.47 %
Principal	W	60	180	0.59 %	4.71 %	1.27 %	0.48 %
Principal	A	30	90	0.75 %	4.43 %	1.66 %	0.38 %
Versiones (comp.)	S	172	860	0.64 %	5.32 %	1.65 %	0.35 %
Versiones (comp.)	W	180	540	0.61 %	4.85 %	1.37 %	0.38 %
Versiones (interp.)	S	180	900	1.59 %	5.88 %	4.16 %	1.27 %

Tabla 4.2: Estabilidad de las repeticiones válidas. CV, rango y diferencia mediana/mejor se calculan por combinación con mediana NAS mayor que cero.

Los casos con mediana NAS igual a 0.00 s se excluyen de los porcentajes relativos de la tabla porque el cociente no está definido. Corresponden a ejecuciones muy cortas de IS clase S en modo 3, donde la resolución de dos decimales de la salida NAS domina la medida. Esos casos sirven para confirmar corrección y orden de magnitud, pero no para ratios finos.

La fórmula de speedup utilizada es

$$\text{speedup}(t) = \frac{\text{mejor_tiempo}(1 \text{ hilo})}{\text{mejor_tiempo}(t \text{ hilos})} \quad (4.1)$$

El speedup se calcula siempre dentro del mismo modo y clase, de forma que el denominador y el numerador comparten el mismo modo de ejecución. Un valor inferior a 1 indica que la ejecución con más hilos fue más lenta que la ejecución secuencial equivalente en ese modo. El límite superior alcanzable está acotado por la fracción secuencial de cada benchmark, de acuerdo con la ley de Amdahl [17]. Las reducciones, barreras y secciones críticas que no se reparten entre hilos fijan el techo de escalabilidad de cada kernel.

Los resultados se clasifican en tres categorías. La primera comprende las ejecuciones correctas y medibles. La segunda agrupa las ejecuciones correctas en las que el overhead domina al trabajo útil. La tercera recoge las no viables, por timeout, error o verificación incorrecta. Esta clasificación separa los límites del algoritmo o del tamaño de clase de los fallos de implementación.

4.7. Amenazas a la validez

La evaluación se diseñó para aislar el efecto del modo de ejecución, el número de hilos y la versión del intérprete, pero conserva varias amenazas a la validez que deben tenerse en cuenta al interpretar los resultados.

4.7.1. Validez interna

El ruido del cluster puede afectar ejecuciones individuales. Para mitigarlo, las campañas usan repeticiones, conservan todos los registros y comparan el mejor tiempo válido dentro de cada combinación. Esta decisión favorece la medida menos perturbada por interferencias externas, y queda respaldada por la baja variabilidad observada entre repeticiones. Además, las campañas finales eliminan por defecto las variables de afinidad de OpenMP para evitar que una política de pinning externa condicione la planificación de hilos del intérprete *free-threaded*.

4.7.2. Validez de medida

El tiempo principal es el tiempo interno NAS, no el tiempo total del proceso. Por eso las conclusiones de rendimiento se refieren al kernel medido por cada benchmark y no al coste de arranque, importación, compilación o verificación. En benchmarks muy cortos, especialmente IS y MG en clase S, la salida NAS con dos decimales produce valores de 0.00 s o 0.01 s. Esos casos sirven para confirmar corrección y tiempos absolutos bajos, pero no para calcular ratios finos. En el profiling con perf stat, los contadores se recogen a nivel de proceso completo. Por tanto, en ejecuciones de centésimas de segundo se usan como apoyo diagnóstico y no como medida exacta del kernel.

4.7.3. Validez externa

Las campañas se ejecutan en nodos clk de Finisterrae III y con un rango máximo de 32 hilos. Los resultados caracterizan esa plataforma y ese rango de concurrencia. Extrapolarlos a otros procesadores, políticas de afinidad o recuentos de hilos requeriría repetir la campaña. La comparación entre versiones se limita a las clases S y W por coste de cluster. La clase A se mide con Python 3.15t en la campaña principal, pero no se repite para Python 3.13t y 3.14t.

4.7.4. Validez del instrumento

El nuevo profiling de Python 3.15 se usa como diagnóstico complementario. La campaña principal y la campaña de perf stat se ejecutan en Finisterrae III. El profiler de Python 3.15 también se ejecuta en Finisterrae III, en clase S y modo 1, para que las primitivas Python del runtime sean visibles al muestreo y al trazado. El trazado completó los cinco benchmarks y proporciona recuentos exactos de regiones, hilos y barreras. El muestreo se usa en los puntos que completaron de forma válida y aporta porcentajes de espera en barreras y reducciones. Estas medidas no sustituyen a los tiempos de la campaña principal, sino que se interpretan como evidencia estructural sobre sincronización, creación de equipos y balance de carga.

Capítulo 5

Resultados

Este capítulo presenta los resultados de la campaña principal ejecutada en Finisterrae III con Python 3.15.0b1t *free-threaded*. La campaña mide tres aspectos, la corrección funcional de los cinco benchmarks portados, el impacto de los modos de ejecución de OMP4Py y la escalabilidad al aumentar el número de hilos.

La campaña cubre CG, EP, FT, IS y MG, y cada clase se midió solo en las configuraciones donde resultaba informativa. La clase S pasó por los cuatro modos, para validar todos los caminos de código. En clase W solo tiene sentido medir los modos compilados 2 y 3, porque los interpretados ya no son representativos a ese tamaño. Y la clase A se reservó al modo 3, porque las clases menores dejan claro que sin tipado Cython no hay tiempos prácticos. Todas las combinaciones usaron 1, 2, 4, 8, 16 y 32 hilos, con tres repeticiones cada una.

La métrica principal es el tiempo NAS interno reportado por cada benchmark. Para cada combinación se toma el mejor tiempo entre las repeticiones válidas. El tiempo externo del proceso se conserva en los ficheros de resultados, pero no se usa como métrica principal porque incluye arranque del intérprete, importación, compilación y escritura de logs.

Campaña	Ejecuciones	Correctas	Éxito	Alcance
Campaña principal Python 3.15t	630	630	100 %	S: modos 0–3; W: modos 2–3; A: modo 3
Comparación de versiones	2340	2340	100 %	Python 3.13t, 3.14t y 3.15t; W: modos 2–3; S: modos 0–3

Tabla 5.1: Cobertura de las campañas finales ejecutadas en Finisterrae III.

La Tabla 5.1 resume la cobertura final. Las 630 ejecuciones de la campaña principal terminaron con código de retorno cero y verificación NAS SUCCESSFUL. Por tanto, el análisis de rendimiento se realiza sobre implementaciones funcionalmente correctas.

5.1. Efecto del modo de ejecución en clase S

La clase S permite comparar los cuatro modos de OMP4Py con bajo coste. El modo 0 conserva la ejecución Python de referencia, el modo 1 aplica la transformación OMP4Py sin

compilar los kernels, el modo 2 genera código Cython sin tipos explícitos y el modo 3 añade tipado Cython y *memoryviews*.

Benchmark	M0 (s)	M1 (s)	M2 (s)	M3 (s)	M0/M3	M2/M3
CG	18.23	18.23	16.35	0.13	140.23×	125.77×
EP	51.97	50.45	42.32	1.44	36.09×	29.39×
FT	38.76	38.70	36.77	2.41	16.08×	15.26×
IS	0.33	0.33	0.30	0.00	—	—
MG	3.81	3.81	3.24	0.01	381.00×	324.00×

Tabla 5.2: Mejor tiempo NAS en clase S con un hilo para los cuatro modos de ejecución en Python 3.15t.

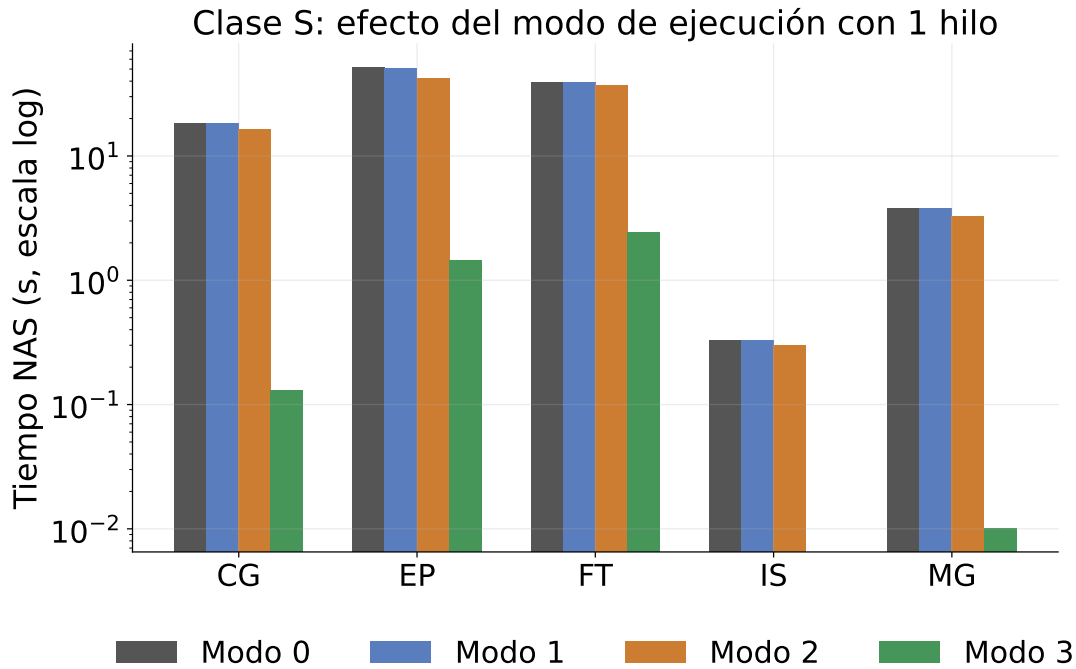


Figura 5.1: Tiempo NAS con un hilo en clase S para los cuatro modos de ejecución. El eje vertical usa escala logarítmica.

La Tabla 5.2 y la Figura 5.1 muestran que el modo 3 obtiene el menor tiempo con un hilo en todos los benchmarks. Frente al modo 0, los factores son 140.23× en CG, 36.09× en EP, 16.08× en FT y 381.00× en MG. IS aparece como 0.00 s en modo 3 porque el tiempo NAS se imprime con dos decimales y la clase S queda por debajo de esa resolución.

La diferencia entre modo 2 y modo 3 confirma que el cuello principal no es solo la compilación, sino el tipado. En CG, el modo 2 tarda 16.35 s y el modo 3 0.13 s. En MG, el modo 2 tarda 3.24 s y el modo 3 0.01 s. EP y FT muestran el mismo patrón con factores de 29.39× y 15.26×.

Sin anotaciones de tipo, Cython conserva demasiada semántica de objetos Python dentro de los bucles críticos. Con el modo 3, esos bucles pasan a ejecutarse como código nativo efectivo.

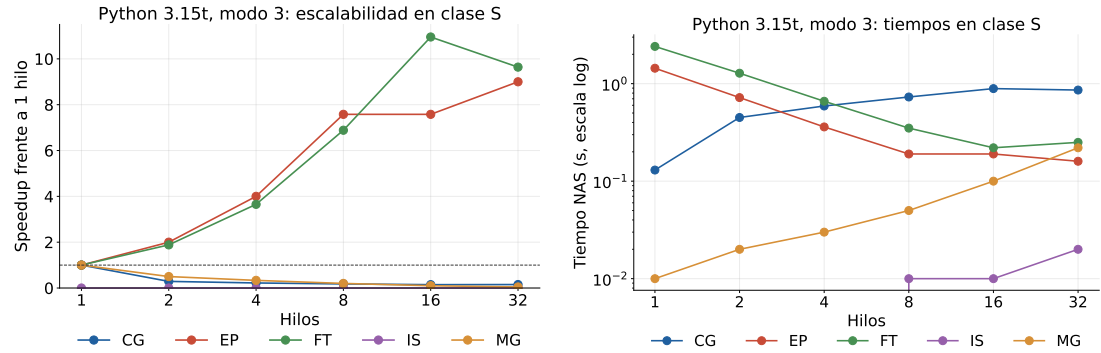
Los modos 0 y 1 quedan prácticamente superpuestos con un hilo. La transformación OMP4Py, por sí sola, no aporta rendimiento cuando no se combina con compilación y tipado. En clase S, su utilidad principal es validar que el código portado mantiene caminos funcionales para todos los modos.

5.2. Escalabilidad en clase S

La clase S no es la carga principal para extraer conclusiones de escalabilidad, pero permite detectar qué kernels tienen suficiente trabajo incluso en tamaños pequeños y qué modos introducen overhead dominante.

Benchmark	M0	M1	M2	M3
CG	0.96×	0.96×	2.34×	0.15×
EP	23.84×	29.16×	24.05×	9.00×
FT	4.00×	4.05×	4.79×	9.64×
IS	1.00×	1.00×	1.25×	—
MG	2.10×	2.23×	8.53×	0.05×

Tabla 5.3: Speedup a 32 hilos respecto a 1 hilo en clase S para cada modo de Python 3.15t.



(a) Speedup. La línea discontinua marca el umbral de speedup 1.

(b) Tiempo NAS por número de hilos.

Figura 5.2: Clase S, modo 3, Python 3.15t. Speedup y tiempo NAS por número de hilos. Cada línea corresponde a un benchmark.

La Tabla 5.3 muestra que EP y FT ya obtienen speedups apreciables a 32 hilos en clase S. En modo 3, EP alcanza 9.00× y FT 9.64×. En EP, los modos 0, 1 y 2 también muestran speedups altos, señal de que el kernel dispone de trabajo paralelo suficiente incluso en la clase pequeña.

El caso de EP en los modos interpretados merece una lectura propia. En modo 1, EP alcanza 29.16× a 32 hilos, una aceleración casi lineal sobre código Python interpretado. Este resultado

es la evidencia más directa de la campaña de que el intérprete *free-threaded* cumple su objetivo. Sin el GIL, los hilos Python ejecutan trabajo en paralelo real sin necesidad de compilación alguna. Que el speedup relativo del modo 3 sea menor ($9.00\times$) no contradice esa lectura. El tipado acelera el cómputo pero no la sincronización, por lo que la fracción de espera pesa más cuanto más rápido es el kernel. La comparación en tiempo absoluto mantiene, no obstante, la conclusión práctica. El modo 3 con un hilo (1.44 s) ya es más rápido que el modo 1 con 32 hilos (aproximadamente 1.7 s), de modo que la escalabilidad casi ideal del modo interpretado tiene valor diagnóstico, no práctico.

El comportamiento no es uniforme. CG y MG empeoran en modo 3, con $0.15\times$ y $0.05\times$ respectivamente. IS no permite calcular un speedup fiable en modo 3 porque el tiempo base aparece redondeado a 0.00 s. La clase S confirma, por tanto, una división que se mantiene en clases mayores. EP y FT son los candidatos naturales a escalar, mientras que CG, IS y MG están más condicionados por sincronización, granularidad o accesos irregulares.

La lectura metodológica es que la clase S sirve para validar todos los caminos y para observar tendencias iniciales, pero las conclusiones prácticas deben apoyarse en W y A. En tamaños tan pequeños, pequeñas diferencias de tiempo o de redondeo pueden afectar mucho a los cocientes.

5.3. Modos compilados en clase W

La clase W aumenta el tamaño del problema y restringe la comparación a los modos 2 y 3. Esta decisión evita dedicar recursos a modos interpretados que ya son varios órdenes de magnitud más lentos en clase S.

Benchmark	M2 (s)	M3 (s)	M2/M3
CG	105.6	0.78	$135.42\times$
EP	84.53	2.87	$29.45\times$
FT	80.07	5.04	$15.89\times$
IS	5.54	0.03	$184.67\times$
MG	198.2	0.45	$440.51\times$

Tabla 5.4: Mejor tiempo NAS en clase W con un hilo para los modos compilados de Python 3.15t.

La Tabla 5.4 mantiene la misma conclusión de clase S. El modo 3 es la única configuración con tiempos prácticos. Con un hilo, el modo 2 tarda 105.6 s en CG frente a 0.78 s en modo 3. En MG la diferencia es de 198.2 s frente a 0.45 s. EP y FT mantienen factores de mejora cercanos a $29.45\times$ y $15.89\times$, e IS mejora de 5.54 s a 0.03 s.

Benchmark	M2	M3
CG	$7.77\times$	$0.14\times$
EP	$26.09\times$	$9.90\times$
FT	$4.37\times$	$8.00\times$
IS	$0.81\times$	$0.43\times$
MG	$23.68\times$	$0.57\times$

Tabla 5.5: Speedup a 32 hilos respecto a 1 hilo en clase W para los modos compilados de Python 3.15t.

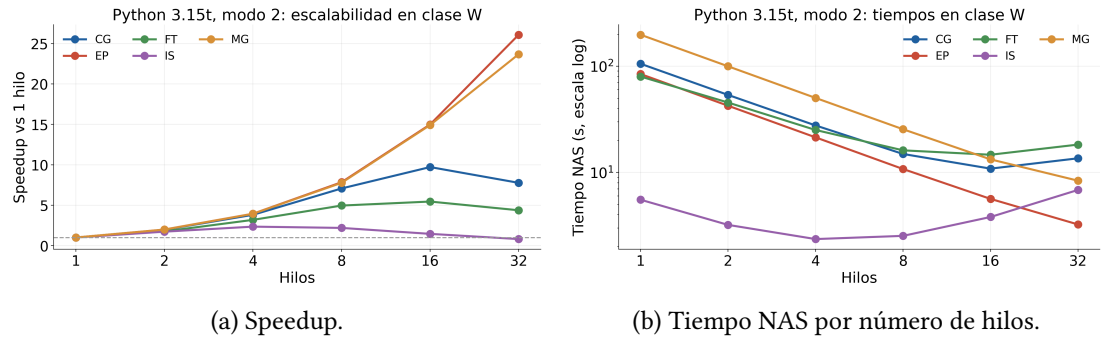


Figura 5.3: Clase W, modo 2, Python 3.15t. Speedup y tiempo NAS por número de hilos.

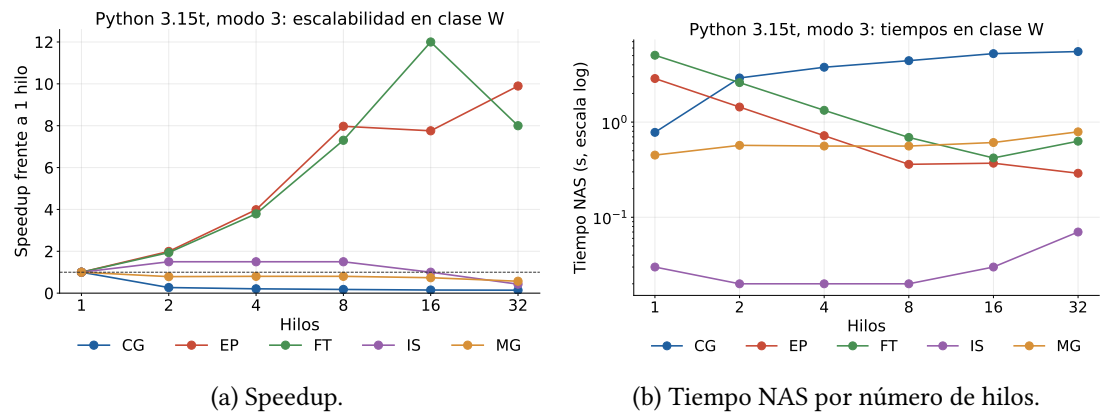


Figura 5.4: Clase W, modo 3, Python 3.15t. Speedup y tiempo NAS por número de hilos. Cada línea corresponde a un benchmark.

La escalabilidad en clase W separa los benchmarks en dos grupos. En modo 3, EP alcanza su mejor tiempo con 32 hilos, bajando de 2.87 s a 0.29 s ($9.90\times$), y FT llega a $8.00\times$. FT obtiene su mejor punto con 16 hilos, donde baja de 5.04 s a 0.42 s, equivalente a $12.00\times$.

CG, IS y MG no escalan en modo 3. CG cae a $0.14\times$, IS a $0.43\times$ y MG a $0.57\times$ a 32 hilos. El modo 2 sí muestra speedups altos en varios de estos casos, por ejemplo $7.77\times$ en CG y $23.68\times$ en MG (Figura 5.3), pero sus tiempos absolutos siguen siendo muy superiores a los del modo 3 con un hilo. El speedup aislado, por sí solo, no basta para escoger configuración. Un modo puede escalar mejor en términos relativos y seguir siendo peor en tiempo absoluto.

5.4. Evaluación de escalabilidad en clase A

La clase A es la carga de mayor tamaño de la campaña principal. Se ejecutó solo en modo 3 porque es la única configuración que produce tiempos prácticos en las clases anteriores.

Benchmark	1	2	4	8	16	32
CG	2.76	10.29	14.35	15.98	18.91	21.17
EP	22.99	11.48	5.75	2.88	2.90	2.19
FT	58.63	30.05	15.26	7.70	3.99	4.18
IS	0.36	0.21	0.15	0.18	0.24	0.44
MG	3.57	4.34	4.14	3.96	3.79	4.56

Tabla 5.6: Mejor tiempo NAS en clase A, modo 3, Python 3.15t.

Benchmark	1	2	4	8	16	32
CG	1.00×	0.27×	0.19×	0.17×	0.15×	0.13×
EP	1.00×	2.00×	4.00×	7.98×	7.93×	10.50×
FT	1.00×	1.95×	3.84×	7.61×	14.69×	14.03×
IS	1.00×	1.71×	2.40×	2.00×	1.50×	0.82×
MG	1.00×	0.82×	0.86×	0.90×	0.94×	0.78×

Tabla 5.7: Speedup por número de hilos en clase A, modo 3, Python 3.15t.

Los resultados de clase A son los más representativos para valorar el port en cargas grandes. EP baja de 22.99 s con un hilo a 2.19 s con 32 hilos, lo que supone $10.50\times$. FT baja de 58.63 s a 4.18 s con 32 hilos ($14.03\times$), y su mejor punto está en 16 hilos con 3.99 s ($14.69\times$). Estos dos benchmarks muestran que Python 3.15t, OMP4Py y el código Cython tipado pueden producir escalabilidad real cuando el kernel ofrece trabajo paralelo suficiente y una estructura adecuada.

IS obtiene una mejora moderada con pocos hilos. Pasa de 0.36 s a 0.15 s con 4 hilos, equivalente a $2.40\times$. Sin embargo, la curva empeora después y a 32 hilos cae a $0.82\times$. El tamaño de clase A permite amortizar parte del coste inicial, pero no sostiene la eficiencia al aumentar la concurrencia.

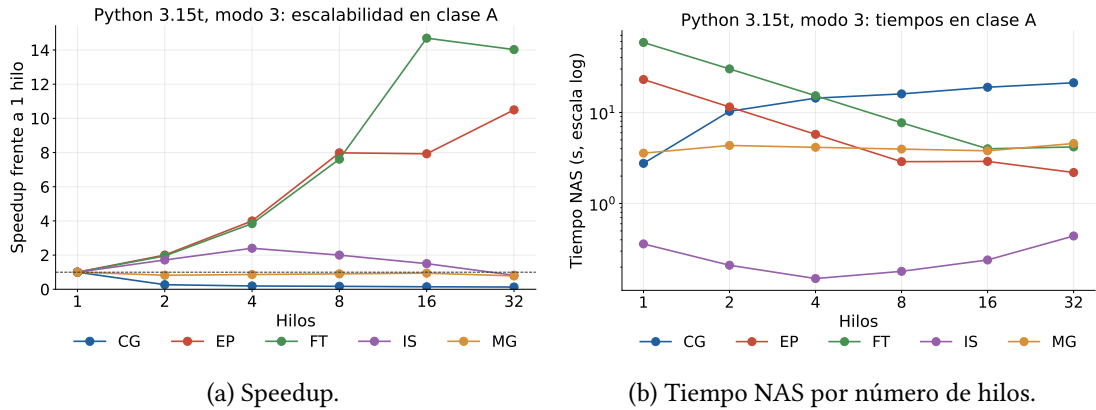


Figura 5.5: Clase A, modo 3, Python 3.15t. Speedup y tiempo NAS por número de hilos. Cada línea corresponde a un benchmark.

CG y MG no escalan en clase A. CG empeora desde 2.76 s con un hilo hasta 21.17 s con 32 hilos. MG sube de 3.57 s a 4.56 s. Aquí se mantiene la lectura de resultados. Más hilos no se traducen en menos tiempo. El Capítulo 6 analiza después las causas de esa degradación.

5.5. Síntesis global del modo 3

Como el modo 3 es el único que produce tiempos competitivos en todas las clases, la síntesis global se centra en esa configuración.

Clase	Bench.	Mejor hilo	Mejor tiempo (s)	T1 (s)	Speedup ópt.	T32 (s)	Speedup32	Mop/s32
S	CG	1	0.13	0.13	1.00×	0.86	0.15×	77.9
S	EP	32	0.16	1.44	9.00×	0.16	9.00×	213
S	FT	16	0.22	2.41	10.95×	0.25	9.64×	702
S	IS	1	0.00	0.00	—	0.02	0.00×	27.7
S	MG	1	0.01	0.01	1.00×	0.22	0.05×	33.9
W	CG	1	0.78	0.78	1.00×	5.51	0.14×	76.3
W	EP	32	0.29	2.87	9.90×	0.29	9.90×	229
W	FT	16	0.42	5.04	12.00×	0.63	8.00×	592
W	IS	2	0.02	0.03	1.50×	0.07	0.43×	158
W	MG	1	0.45	0.45	1.00×	0.79	0.57×	618
A	CG	1	2.76	2.76	1.00×	21.17	0.13×	70.7
A	EP	32	2.19	22.99	10.50×	2.19	10.50×	245
A	FT	16	3.99	58.63	14.69×	4.18	14.03×	1706
A	IS	4	0.15	0.36	2.40×	0.44	0.82×	191
A	MG	1	3.57	3.57	1.00×	4.56	0.78×	854

Tabla 5.8: Mejor número de hilos, speedup y rendimiento (Mop/s) a 32 hilos en modo 3, Python 3.15t.

La Tabla 5.8 resume el mejor número de hilos en cada combinación y recoge además los tiempos y los Mop/s a 32 hilos, mientras que la Figura 5.6 muestra de forma compacta qué

combinaciones escalan a 32 hilos y cuáles retroceden. EP alcanza su mejor punto en 32 hilos en las tres clases, con $9.00\times$ en S, $9.90\times$ en W y $10.50\times$ en A. FT alcanza su mejor punto en 16 hilos, con $10.95\times$ en S, $12.00\times$ en W y $14.69\times$ en A. IS solo muestra una mejora clara en A hasta 4 hilos. CG y MG mantienen el óptimo en un hilo en las tres clases.

Los Mop/s a 32 hilos refuerzan esa lectura. FT clase A alcanza 1706 Mop/s y EP clase A 245 Mop/s con speedups superiores a $10\times$. En MG, en cambio, la cifra de 854 Mop/s a 32 hilos no implica escalabilidad, porque el tiempo empeora frente a un hilo. De ahí que el análisis combine siempre tiempo absoluto, speedup y Mop/s.

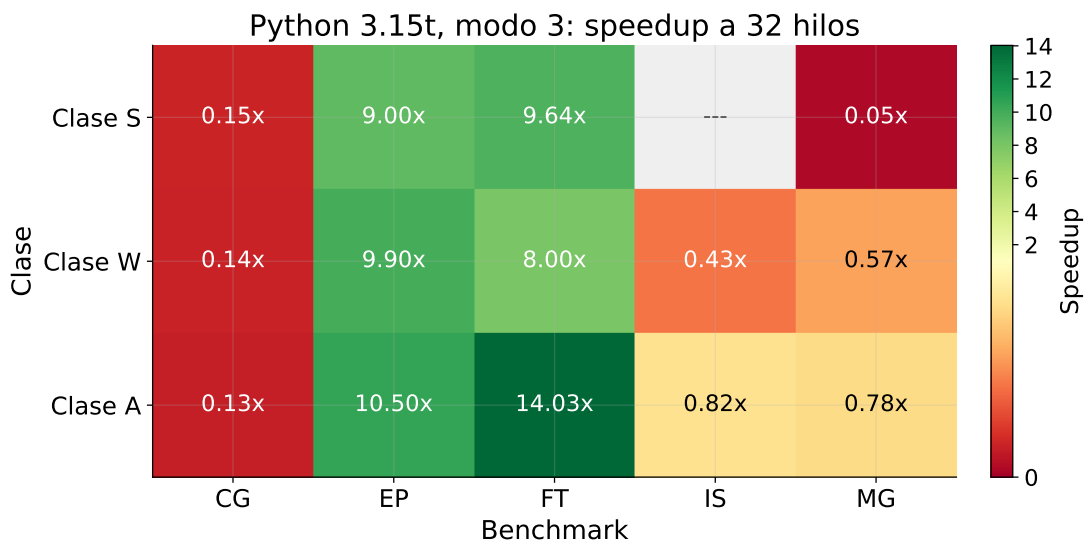


Figura 5.6: Mapa de calor del speedup a 32 hilos en modo 3, Python 3.15t, para las clases S, W y A.

5.6. Curvas por benchmark

La Figura 5.7 reorganiza la misma información por benchmark. En cada panel, cada línea corresponde a una clase de problema. Esta vista facilita comparar si el aumento de tamaño cambia la forma de la curva de escalabilidad.

La lectura por benchmark confirma la separación observada en las tablas. EP mantiene una curva creciente en S, W y A. FT también mejora con el tamaño, aunque su máximo aparece antes de 32 hilos. IS solo se beneficia de forma clara en clase A y con pocos hilos. CG y MG no mejoran al aumentar la clase. En este punto, los resultados muestran el patrón, y el Capítulo 6 analiza sus causas mediante perf stat, muestreo y trazado de Python 3.15.

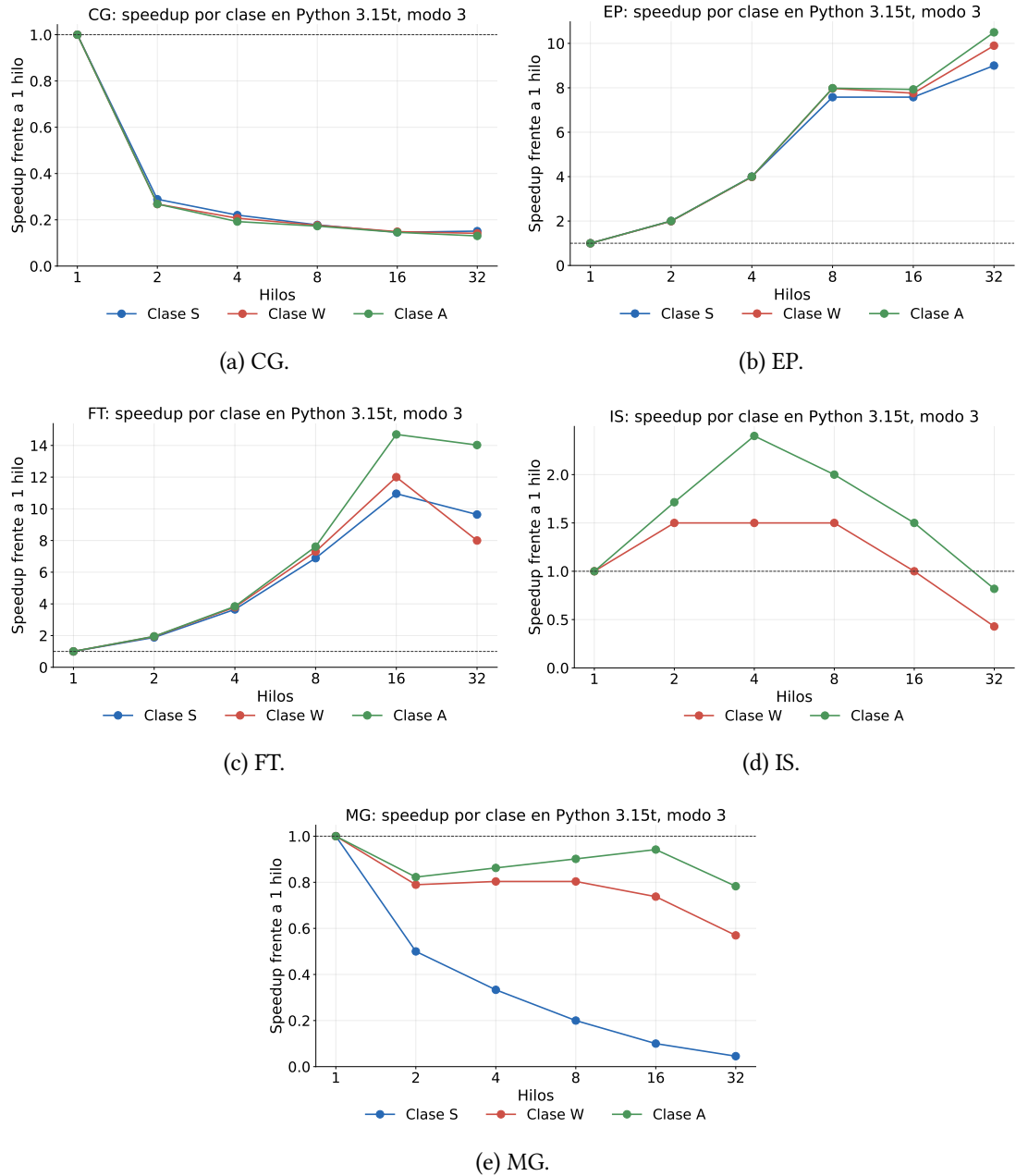


Figura 5.7: Speedup por clase (S, W, A) en modo 3, Python 3.15t, para cada benchmark.

5.7. Limitaciones de la medida

Las amenazas transversales de la evaluación se recogen en la Sección 4.7. En la campaña principal afectan sobre todo a tres puntos. El primero es la resolución temporal de la salida NAS. Algunos tiempos de clase S, especialmente en IS y MG, aparecen como 0.00 s o 0.01 s. Sirven para confirmar tiempos absolutos bajos, pero no para calcular ratios finos.

El segundo es el uso del mejor tiempo de tres repeticiones como métrica principal de rendimiento. La Sección 4.6 contrasta esa elección con mediana y rango, y muestra que la diferencia mediana entre la mediana y el mejor tiempo queda por debajo del 0.5 % en las campañas compiladas y se mantiene baja en los modos interpretados. No sustituye a una campaña estadística más amplia, pero sí descarta que las conclusiones dependan de una repetición aislada.

El tercero es que la clase A solo se ejecutó en modo 3. No hay medidas de modo 2 para A en la campaña final. La restricción fue deliberada porque el modo 2 ya había demostrado tiempos demasiado altos en clase W. Las conclusiones de clase A se refieren, pues, a la escalabilidad del mejor modo disponible.

5.8. Conclusiones del capítulo

La campaña principal deja un balance claro. Las cinco adaptaciones son funcionalmente correctas en todos los caminos evaluados, y el modo 3 recorta de forma decisiva los tiempos del modo 2. Con Python 3.15t, EP y FT escalan bien en las clases mayores, pero esa escalabilidad no se generaliza a CG, IS y MG.

El Capítulo 6 utiliza `perf stat` para relacionar los speedups observados con ocupación real de CPU, IPC y eventos de memoria, y el nuevo sistema de muestreo de Python 3.15 para atribuir el tiempo de sincronización y el balance de carga por función e hilo. El Capítulo 7 compara estos resultados con Python 3.13t y Python 3.14t.

Profiling y diagnóstico

6.1. Objetivo del profiling

Los capítulos anteriores establecen dos hechos. El modo 3 reduce de forma marcada los tiempos frente al modo 2, y solo EP y FT obtienen aceleración sostenida. El objetivo de este capítulo es relacionar esos resultados con métricas de ejecución de bajo nivel, como la ocupación de CPU, el IPC y los eventos de caché. El diagnóstico se apoya en dos herramientas. La primera es `perf stat` [16], que mide el uso de recursos a nivel de proceso. La segunda es el nuevo sistema de muestreo de Python 3.15 (Sección 6.5), que atribuye el tiempo de sincronización y el balance de carga por función e hilo.

El profiling con `perf stat` mide el proceso completo y, por tanto, no aísla únicamente la ventana temporizada por el benchmark NAS. En este capítulo se usa una matriz común para todos los benchmarks, con Python 3.15t, clase W, modo 3 y 1, 8 y 32 hilos. Así, la lectura de CPU, IPC y caché no mezcla efectos de versión con diferencias de diseño de la campaña. Cada medición incluyó una ejecución de calentamiento previa para reducir el efecto de la compilación Cython sobre los contadores.

Las métricas de `perf stat` deben interpretarse con cautela en benchmarks cortos. IS clase W tarda centésimas de segundo. En ese caso, el arranque del proceso y las bibliotecas importadas pueden pesar tanto como el kernel. También se observan valores de CPUs utilizadas superiores a uno en algunas ejecuciones con un hilo, precisamente porque `perf` mide el proceso completo y no solo la región paralela.

6.2. Control de entorno y afinidad

Las campañas finales no fijan por defecto las variables de afinidad de OpenMP (`OMP_PROC_BIND`, `OMP_PLACES` y `GOMP_CPU_AFFINITY`), y las eliminan de forma explícita en los scripts de ejecución salvo en experimentos controlados. El motivo es un efec-

to que conviene precisar, porque no nace de OMP4Py. La biblioteca no fija hilos a núcleos. Lee `OMP_PROC_BIND` a una variable interna sin aplicarla y la cláusula `proc_bind` no está soportada. El anclaje lo introduce el entorno a través de una librería nativa. En Finisterrae III, NumPy carga por debajo libgomp, y cuando el entorno define `OMP_PROC_BIND` y `OMP_PLACES`, libgomp restringe la máscara de CPU del proceso a un único núcleo durante el propio import de NumPy, antes de que OMP4Py cree ningún hilo. Como los intérpretes *free-threaded* heredan la máscara de CPU del proceso, todos los hilos que crea OMP4Py quedan confinados a ese núcleo y el cómputo se serializa. El efecto aparece en cualquier modo de ejecución, porque se produce al importar NumPy y no en una capa concreta de la biblioteca. Al eliminar esas variables, la máscara abarca todos los núcleos asignados y EP y FT vuelven a escalar, de forma comparable en Python 3.13t, 3.14t y 3.15t.

Una comprobación controlada lo confirma. Cambiando solo esas variables sobre el mismo binario y el mismo nodo, EP en modo 3 y clase S se mantiene en torno a 1.44 s de 1 a 32 hilos cuando están activas y baja a 0.19 s al eliminarlas, mientras la máscara medida pasa de los 32 núcleos de la asignación a uno solo en el import de NumPy. El mismo patrón se repite en los cuatro modos de ejecución. El planificador de Slurm no define esas variables por su cuenta, de modo que en las campañas preliminares procedían del entorno de ejecución.

También se realizaron pruebas de aislamiento. `sys._is_gil_enabled()` se mantuvo en False antes y después de importar OMP4Py, Cython y los módulos compilados. Además, un microbenchmark con hilos Python puros y un kernel OMP4Py mínimo usaron varias CPUs en Python 3.14t y Python 3.15t. Estas pruebas apoyan la interpretación de que los problemas restantes pertenecen a los kernels concretos o a su interacción con el runtime, no a una reactivación implícita del GIL.

6.3. Diagnóstico común con perf stat

La Tabla 6.1 separa tres comportamientos. EP y FT reducen el tiempo al añadir hilos. En EP, el tiempo baja de 2.87 s a 0.37 s con 8 hilos, y en FT baja de 4.90 s a 0.70 s. La ocupación de CPU confirma que ambos kernels ejecutan trabajo paralelo real. FT alcanza 3.57 CPUs con 8 hilos y 5.72 CPUs con 32. En FT, sin embargo, el IPC cae y los fallos de caché suben a 32 hilos, lo que encaja con el óptimo observado alrededor de 16 hilos en la campaña principal.

CG y MG muestran el patrón contrario. Activan varias CPUs, pero el tiempo empeora. CG pasa de 0.78 s con un hilo a 4.59 s con 8 y 6.42 s con 32. Al mismo tiempo, las CPUs usadas suben hasta 5.98 y el IPC cae de 3.07 a 0.39. Por tanto, el proceso no está serializado, sino que ejecuta paralelismo ineficiente. MG es menos extremo, pero sigue la misma dirección. Usa 3.04 CPUs con 8 hilos y 3.95 CPUs con 32, mientras el tiempo aumenta y el IPC baja.

Bench.	Hilos	NAS (s)	Speedup	CPUs usadas	IPC	Fallos cache (%)
EP	1	2.87	1.00×	0.97	1.09	14.8
EP	8	0.37	7.76×	2.47	1.02	44.8
EP	32	0.48	5.98×	2.55	1.09	18.9
FT	1	4.90	1.00×	0.97	2.71	14.0
FT	8	0.70	7.00×	3.57	2.49	30.0
FT	32	0.64	7.66×	5.72	1.88	44.2
CG	1	0.78	1.00×	0.99	3.07	6.7
CG	8	4.59	0.17×	2.41	1.04	3.0
CG	32	6.42	0.12×	5.98	0.39	44.5
IS	1	0.03	1.00×	0.89	1.93	25.2
IS	8	0.03	1.00×	1.08	1.90	23.1
IS	32	0.08	0.38×	1.23	1.83	25.8
MG	1	0.46	1.00×	0.93	2.86	25.0
MG	8	0.60	0.77×	3.04	0.85	39.8
MG	32	0.93	0.49×	3.95	0.65	40.7

Tabla 6.1: Perfil homogéneo con perf stat en Finisterrae III: Python 3.15t, clase W, modo 3 y recuentos de 1, 8 y 32 hilos para todos los benchmarks.

IS debe leerse con más cautela porque sus tiempos son de centésimas de segundo. La señal de rendimiento sigue siendo útil. Con 32 hilos el tiempo sube a 0.08 s, y el trabajo por hilo no compensa la coordinación, los histogramas y las fases de acumulación. Sus contadores de CPU, en cambio, son menos informativos porque el arranque del proceso pesa mucho respecto a la ventana del kernel.

6.4. Interpretación conjunta

El diagnóstico con perf stat distingue ya tres comportamientos. EP y FT usan varias CPUs y reducen el tiempo. CG y MG activan varias CPUs, pero el IPC cae al aumentar los hilos y el tiempo empeora. IS es demasiado corto para sostener trabajo entre sincronizaciones. La Sección 6.5 cruza esta lectura, benchmark a benchmark, con el muestreo y el trazado de Python 3.15.

La conclusión del diagnóstico es que los malos speedups no tienen una única causa. En las campañas preliminares, la afinidad heredada del entorno que se describe en la Sección 6.2 falseaba la lectura de Python 3.15t. En las finales ese efecto desaparece en EP y FT, y los límites restantes dependen de la estructura de cada benchmark y de la eficiencia del runtime en regiones con sincronización o granularidad difícil.

6.5. Profiling con el nuevo sistema de Python 3.15

El análisis con `perf stat` caracteriza el uso de recursos a nivel de proceso (CPUs, IPC, caché), pero no descompone el tiempo entre trabajo útil y sincronización dentro del runtime. Python 3.15 incorpora un nuevo sistema de profiling en la biblioteca estándar, apoyado en la interfaz de depuración externa de CPython [18], con dos componentes. El de muestreo, `profiling.sampling`, toma muestras estadísticas de las pilas de llamadas de todos los hilos del proceso [19]. En modo *wall*, un hilo bloqueado en una barrera o una reducción aparece en la muestra dentro de su función de espera, lo que permite atribuir tiempo a la sincronización por nombre de función. El de trazado, `profiling.tracing`, es un profiler determinista que registra cada llamada y devuelve recuentos exactos por función. Esta sección usa el muestreo para repartir el tiempo entre cómputo y sincronización, y el trazado para contar cuántas veces se ejecutan las operaciones del runtime, dos vistas que `perf stat` solo sugiere de forma indirecta.

6.5.1. Entorno y metodología

La campaña con el profiler de Python 3.15 se ejecutó en Finisterrae III con Python 3.15.0b1t *free-threaded*, clase S y modo 1. Se escogió modo 1 (híbrido, hilos Python reales sobre el runtime interpretado) porque en ese modo las primitivas de sincronización de OMP4Py son funciones Python visibles al profiler. En modo 3 los kernels y sus barreras se compilan a código nativo y el muestreador de pilas Python no las observa. Cada ejecución de muestreo se lanzó en modo *wall*, con muestreo de todos los hilos y una frecuencia de 2000 Hz.

El trazado completó los cinco benchmarks con 8 hilos y proporciona los recuentos estructurales de regiones, hilos creados y barreras. El muestreo aporta una segunda vista, basada en muestras de pila, para los puntos que completaron de forma válida en Finisterrae III. Las campañas de rendimiento de los capítulos anteriores siguen siendo la referencia para tiempos y speedups. El profiling de Python 3.15 se usa para identificar patrones estructurales del runtime, como espera en barreras, reducciones, creación de equipos y reparto de trabajo entre hilos.

6.5.2. Del muestreo a las tablas

El muestreador imprime una tabla de funciones ordenada por número de muestras. La obtenida para EP con 8 hilos, recortada en ancho, es la siguiente.

```
$ python3.15t -m profiling.sampling run --mode wall -r 2000 \  
  examples/EP_Python.py -c S -t 8 -m 1
```

Profile Stats:

```
nsamples sample% tottime(ms) filename:lineno(function)
```

1461	7.9	730.5	EP_Python.py:214(...__omp_parallel)
1351	7.3	675.5	threading.py:363(Condition.wait)
1331	7.2	665.5	EP_Python.py:210(...__omp_parallel)
1022	5.5	511.0	EP_Python.py:136(vranlc_ep)
...			

Las filas dentro del cuerpo de la región (`__omp_parallel`) y de `vranlc_ep` son cómputo y suman alrededor del 70 % de las muestras. La fila de `threading.Condition.wait` (7.3 %) es la espera en la barrera del runtime. Para las tablas de este apartado, que necesitan la atribución por hilo, las ejecuciones se relanzaron con `-collapsed`, que en lugar de la tabla agregada vuelca una pila por muestra con su hilo. Un script clasifica entonces la hoja de cada pila como cómputo, sincronización o inicialización. La fracción de sincronización de la Tabla 6.2 sale del cociente entre las muestras de espera y la suma de espera y cómputo. El número de hilos de la Tabla 6.3 es la cantidad de identificadores de hilo distintos que aparecen en esos volcados.

6.5.3. Coste de sincronización

La Tabla 6.2 mide la fracción de las muestras de ejecución paralela que caen en primitivas de espera del runtime. El hilo que espera aparece en `threading.Condition.wait`, alcanzada a través de `parallelism.py:parallel_run` y la barrera interna del runtime. En EP, la espera es muy baja con pocos hilos y sube al 6.9 % con 16 hilos. En CG, FT y MG la fracción de sincronización ya es mayor con 4 hilos y aumenta al subir a 16 hilos. IS es el caso más extremo. Al tener poco trabajo útil por región, la espera alcanza casi la mitad del tiempo paralelo con 16 hilos. Este patrón es coherente con la ley de Amdahl [17] y con los resultados de rendimiento. El tipado del modo 3 acelera el cómputo pero no elimina la coordinación, de modo que la fracción de espera observada aquí señala el límite efectivo de escalabilidad cuando el kernel es rápido.

Benchmark	1 hilo	4 hilos	16 hilos
EP	0.0 %	0.4 %	6.9 %
FT	0.0 %	1.9 %	5.5 %
CG	0.2 %	3.8 %	14.2 %
IS	0.2 %	9.8 %	49.3 %
MG	0.1 %	3.6 %	10.8 %

Tabla 6.2: Coste de sincronización medido con `profiling.sampling` en Finisterrae III. Porcentaje de muestras de espera sobre ejecución paralela; clase S, modo 1.

6.5.4. Gestión de hilos y balance de carga

El profiler revela además una diferencia estructural que `perf stat` no expone (Tabla 6.3). EP ejecuta toda su región paralela sobre un único equipo persistente de hilos, y con 16 hilos

se observan 15 hilos trabajadores distintos. CG, FT, IS y MG, en cambio, crean y destruyen equipos entre regiones paralelas, de modo que a lo largo de una ejecución aparecen decenas, cientos o miles de identificadores de hilo, 179 en IS, 512 en FT, 740 en MG y 6797 en CG. Esa creación y destrucción repetida de equipos es en sí misma una fuente de sobre coste y explica parte de la degradación al aumentar la concurrencia. Sobre el equipo persistente de EP, el reparto de muestras de cómputo entre hilos trabajadores es prácticamente equilibrado. En los kernels con rotación de equipos no existe un equipo estable sobre el que medir balance, y el propio recuento de hilos efímeros pasa a ser el indicador relevante. El origen de esa rotación está en el diseño del runtime, que se analiza en la Sección 6.6.

Benchmark	Hilos trabajadores distintos	Sincr./paralelo
EP	15 (equipo persistente)	6.9 %
FT	512	5.5 %
CG	6797	14.2 %
IS	179	49.3 %
MG	740	10.8 %

Tabla 6.3: Gestión de hilos medida con `profiling.sampling` en Finisterrae III. Hilos trabajadores distintos observados con 16 hilos; clase S, modo 1.

6.5.5. Recuento exacto con el profiler de trazado

El muestreo reparte el tiempo, pero no cuenta cuántas veces ocurre cada operación. Ese recuento lo aporta el componente de trazado, `profiling.tracing`, un profiler determinista que registra cada llamada. Su instrumentación infla los tiempos absolutos, así que aquí solo se usan los recuentos de llamadas, que son exactos. Sobre CG en clase S, modo 1 y 8 hilos, el trazado contabiliza 1682 regiones paralelas, 11 774 creaciones de hilo (`threading.Thread`) y 27 168 entradas en la barrera del runtime (`task_barrier`). El muestreo observa miles de identificadores de hilo, pero el trazado da el número real, porque captura también los hilos demasiado breves para aparecer en una muestra. EP, con una sola región paralela, crea su equipo una única vez y reutiliza 7 hilos durante todo el cómputo. La diferencia entre 7 hilos y casi 12 000 pone cifra al sobrecoste de crear y destruir un equipo por región, y confirma que el límite de CG está en el runtime, no en el cómputo del kernel.

La Tabla 6.4 extiende el recuento a los cinco benchmarks. El número de regiones crece de 1 en EP a 12 en IS, 40 en FT, 127 en MG y 1682 en CG, y los hilos creados lo acompañan, de 7 a 11 774. Ese gradiente coincide con el orden de escalabilidad observado en los capítulos anteriores.

Benchmark	Regiones	Hilos creados	Barreras
EP	1	7	16
FT	40	280	640
CG	1682	11774	27168
IS	12	84	184
MG	127	889	3936

Tabla 6.4: Recuentos exactos del runtime medidos con `profiling.tracing` en Finisterrae III. Clase S, modo 1 y 8 hilos.

6.5.6. Síntesis por benchmark

Cruzando las herramientas, cada benchmark tiene un perfil propio. Las medidas con `perf stat` corresponden a clase W y modo 3. Las del muestreo y el trazado corresponden a clase S y modo 1 en Finisterrae III, de modo que se leen como diagnósticos complementarios y no como una única serie temporal.

EP escala. En clase W, `perf` mide entre 3 y 5 CPUs activas y un IPC estable, sin fallos de caché destacables. En clase S, el muestreo registra poca espera a baja concurrencia y el trazado lo explica. Abre una sola región y reutiliza un equipo de 7 hilos durante todo el cómputo, sin rotación. Su único límite a recuentos altos es el coste fijo de esa región, no la estructura del kernel.

FT escala con meseta. `perf` muestra varias CPUs activas, pero el IPC baja y los fallos de caché suben a 32 hilos, coherente con una FFT que combina cómputo y accesos a memoria. El

muestreo ve espera por sincronización en las pasadas paralelas, y el trazado cuenta 40 regiones y 280 hilos creados. El óptimo medido está en 16 hilos.

CG no escala. perf llega a activar unas 6 CPUs, pero el IPC se desploma de 3.07 con un hilo a 0.39 con 32 y el tiempo empeora. El muestreo atribuye cerca del 14 % del tiempo paralelo a espera con 16 hilos, y el trazado descubre la causa estructural, con 1682 regiones y 11 774 hilos creados en una sola ejecución de clase S. El cuello es la coordinación, no el cómputo.

IS es un caso de trabajo insuficiente. Su kernel es tan corto que perf resulta ruidoso. El trazado muestra pocas regiones (12) y pocos hilos (84), así que el problema no es la rotación de equipos, sino que hay tan poco trabajo por región que el coste de coordinar varios hilos lo supera enseguida.

MG no escala, y combina dos efectos. perf mide 3 o 4 CPUs activas pero el tiempo sube, con el IPC a la baja y más fallos de caché. El trazado cuenta 127 regiones y 889 hilos creados, una rotación intermedia entre IS y CG, y a ella se añade la granularidad desigual de los niveles de malla, donde los niveles gruesos activan un equipo para apenas 1 o 2 iteraciones útiles.

En conjunto, el muestreo y el trazado de Python 3.15 llevan el diagnóstico desde los síntomas agregados de perf stat hasta una atribución por función, hilo y recuento, que es la base de las mejoras de runtime de la Sección 6.6.

6.6. Implicaciones para OMP4Py

El diagnóstico no invalida el port. EP y FT muestran que las regiones paralelas adaptadas con OMP4Py producen speedups altos cuando el kernel es adecuado, y el modo 3 confirma que el tipado Cython elimina gran parte del coste de objetos Python en los bucles críticos. Más allá del comportamiento de cada benchmark, el nuevo sistema de profiling de Python 3.15 señala dos puntos del propio runtime que limitan la escalabilidad con independencia del kernel, y que son los candidatos más claros a mejora en la biblioteca. El análisis se hizo sobre el runtime interpretado. Como el runtime nativo conserva la misma estructura general de equipos, barreras y reducciones, es razonable esperar que parte de estos límites se mantenga en los modos compilados. La magnitud exacta, sin embargo, requiere instrumentación interna del runtime nativo.

6.6.1. Reutilización del equipo de hilos

El primer límite es la gestión del equipo de hilos. En su versión actual, el runtime crea un equipo nuevo en cada región paralela. La función `parallel_run` lanza un `threading.Thread` por trabajador y lo descarta al cerrar la región, sin un pool persistente que se reutilice entre regiones. El coste de crear y destruir hilos es fijo por región, de modo que un kernel con una sola región (EP) lo paga una vez, mientras que un kernel con muchas regiones cortas (CG, o

el ciclo en V de MG) lo paga miles de veces. Es la diferencia entre el equipo estable de EP y los miles de identificadores de hilo que aparecen en CG (Tabla 6.3). De hecho, los benchmarks que escalan se reescribieron para envolver todo el cómputo en una única región parallel y repartir el trabajo interno con for, justamente para evitar esa rotación. Que esa reescritura sea necesaria indica que la mejora corresponde al runtime. Un equipo persistente, creado una vez y aparcado en una barrera entre regiones, eliminaría el sobre coste sin obligar al usuario a reestructurar su código.

6.6.2. Coste de la barrera y la reducción

El segundo límite es el coste de la sincronización. Cada región termina en una barrera (`task_barrier`) construida sobre `threading.Event`, que a su vez bloquea en `threading.Condition.wait`. Esas primitivas de la biblioteca estándar son relativamente pesadas, y su coste se nota cuando el trabajo entre barreras es pequeño. La fracción de tiempo en espera alcanza valores relevantes con 16 hilos, especialmente en CG, IS y MG (Tabla 6.2). Como cada reducción escalar conlleva además una barrera, un kernel con varias reducciones por iteración, como las cuatro de CG, acumula ese coste. Una barrera más ligera, por ejemplo una barrera centralizada con contador o una implementación con operaciones atómicas en el runtime nativo, y una reducción que no exija una barrera completa, reducirían esa fracción de espera.

6.6.3. Calidad del código compilado

El tercer nivel es la calidad del código que genera el modo compilado. Si en el bucle crítico queda una llamada a una función Python externa, Cython no puede compilarla y el tiempo absoluto vuelve a estar dominado por el intérprete, como ocurría en EP con `vranlc` antes de incluir una versión compilable por Cython (Sección 3.3). Mantener todo el cuerpo del kernel compilable es, por tanto, tan importante como el tipado.

Estas observaciones provienen de profiling externo sobre el runtime interpretado en Finisterrae III. El siguiente paso sigue siendo instrumentar OMP4Py internamente, por región y para los modos compilados, midiendo cuántos hilos entran en cada región y qué fracción de tiempo pasan en trabajo útil frente a creación de equipos, barreras y reducciones. Eso convertiría estas implicaciones en una guía cuantitativa para el desarrollo de la biblioteca.

Comparación entre versiones de Python *free-threaded*

El capítulo anterior muestra que, en la campaña principal con Python 3.15t, el modo 3 de OMP4Py produce tiempos competitivos y escala en EP y FT, mientras que CG, IS y MG presentan límites de paralelización. Este capítulo estudia si ese comportamiento depende de la versión concreta del intérprete *free-threaded*, y recorre para ello los cuatro modos de ejecución. La respuesta tiene dos caras. En los modos interpretados la versión importa mucho, porque la primera build *free-threaded*, la 3.13t, escala bastante peor al aumentar los hilos. En los modos compilados esa diferencia desaparece. Por eso el capítulo arranca por los modos interpretados, donde el efecto es mayor, y después comprueba que el tipado lo anula.

La comparación cubre Python 3.13.13t, Python 3.14.4t y Python 3.15.0b1t, todas compiladas con el GIL desactivado. Las campañas finales se ejecutaron con el mismo código fuente, la misma infraestructura de Slurm, las mismas clases y el mismo rango de hilos. Además, se eliminaron las variables de afinidad de OpenMP que habían producido resultados preliminares no representativos. Por tanto, la variable experimental de este capítulo es la versión del intérprete y su ABI *free-threaded*.

7.1. Diseño de la campaña

La campaña de comparación entre versiones se diseñó como un subconjunto de la campaña principal, pero manteniendo todas las potencias de dos hasta 32 hilos. Se ejecutaron CG, EP, FT, IS y MG con los recuentos de hilos 1, 2, 4, 8, 16 y 32. Los modos compilados 2 y 3 se midieron en las clases S y W, y los modos interpretados 0 y 1 en la clase S, el único tamaño donde el código no compilado termina en un tiempo razonable. La clase S se ejecutó con cinco repeticiones por combinación y la clase W con tres. La campaña completa contiene 2340 eje-

cuciones, 1440 en los modos compilados y 900 en los interpretados, y todas finalizaron con verificación SUCCESSFUL.

No se incluyó la clase A porque el coste habría multiplicado el uso del cluster. La clase A ya se evaluó con Python 3.15t en la campaña principal, y esta campaña se centra en comprobar si las tendencias de S y W cambian al modificar el intérprete. Como en el resto de la memoria, se usa el mejor tiempo NAS entre las repeticiones válidas.

7.2. Efecto de la versión en los modos interpretados

La comparación empieza por los modos 0 y 1, donde los kernels se ejecutan como Python interpretado y es el propio intérprete el que realiza el trabajo. La Tabla 7.1 recoge el tiempo NAS de los cinco benchmarks en los dos modos interpretados, para las tres versiones y todos los recuentos de hilos, y la Figura 7.1, al final de la sección, lo muestra de forma gráfica.

Bench.	Hilos	Modo 0			Modo 1		
		3.13t	3.14t	3.15t	3.13t	3.14t	3.15t
CG	1	18.35	18.21	18.24	18.33	18.15	18.28
	2	18.07	14.85	14.28	18.46	15.26	14.77
	4	18.30	10.23	11.12	17.79	10.31	10.92
	8	31.95	12.60	12.96	32.31	12.36	12.61
	16	90.49	14.71	14.72	92.82	14.15	14.57
	32	115.78	22.36	24.15	100.69	16.75	16.61
EP	1	59.56	52.33	50.88	60.16	53.68	52.02
	2	29.73	26.31	25.88	30.67	27.44	26.53
	4	15.26	13.27	12.96	15.56	13.90	13.45
	8	7.57	6.72	6.52	7.86	7.11	6.71
	16	4.08	5.13	3.39	3.87	4.10	3.35
	32	2.10	3.58	1.74	2.24	3.50	2.05
FT	1	42.88	47.50	39.32	39.97	47.54	39.21
	2	25.17	25.98	22.65	24.92	25.98	22.85
	4	15.12	14.69	12.87	15.37	14.49	12.90
	8	10.70	9.35	8.34	10.92	9.43	8.27
	16	14.38	8.18	7.56	15.15	8.17	7.55
	32	17.85	8.63	8.16	18.21	8.61	8.17
IS	1	0.33	0.32	0.33	0.33	0.32	0.33
	2	0.28	0.22	0.22	0.29	0.22	0.22
	4	0.25	0.15	0.15	0.23	0.15	0.15
	8	0.39	0.17	0.19	0.39	0.17	0.19
	16	1.01	0.24	0.25	1.02	0.24	0.25
	32	1.23	0.31	0.32	1.26	0.32	0.33
MG	1	3.69	3.70	3.80	3.69	3.67	3.80
	2	3.13	2.38	2.52	3.04	2.29	2.47
	4	2.75	1.52	1.63	2.73	1.53	1.62
	8	3.32	1.26	1.32	3.16	1.27	1.29
	16	8.63	1.28	1.30	8.54	1.30	1.32
	32	10.65	1.43	1.45	10.62	1.42	1.45

Tabla 7.1: Tiempo NAS por número de hilos en los modos interpretados 0 y 1, clase S, para Python 3.13t, 3.14t y 3.15t. Tiempos en segundos. Al crecer el número de hilos, Python 3.13t se degrada mucho más que 3.14t y 3.15t en CG, FT, IS y MG. EP, *embarrassingly parallel*, es la excepción.

Con un solo hilo, CG, IS y MG quedan casi idénticos entre versiones, porque su cómputo se apoya en NumPy y la ruta caliente corre en C. Los que se separan son EP y FT, los dos kernels

con más trabajo en el propio intérprete. En EP, Python 3.13t resulta entre un 12 % y un 17 % más lento que 3.14t y 3.15t, coherente con que la 3.13t desactivara el intérprete especializado por no ser todavía seguro sin GIL, función que 3.14t recuperó. En FT el margen es aún mayor, en torno al 20 %, y con otro orden, porque la más lenta es 3.14t.

El cuadro cambia por completo al aumentar el número de hilos (Figura 7.1). En CG, FT, IS y MG, Python 3.13t se degrada mucho más que 3.14t y 3.15t, y la brecha crece con cada duplicación de hilos. CG en modo 1 pasa de 18.3 s con un hilo, donde las tres versiones coinciden, a 100.7 s con 32 hilos en 3.13t frente a 16.6 s en 3.15t, seis veces más. MG llega a ser siete veces más lento en 3.13t a 32 hilos (10.6 s frente a 1.4 s), IS unas cuatro veces y FT algo más del doble. Con un hilo la diferencia no existe, con 8 hilos ya es de dos a tres veces, y con 32 alcanza de cuatro a siete veces. Es un efecto sistemático, no ruido, porque se repite en los dos modos interpretados y en las cinco repeticiones de cada punto.

EP es la única excepción. Al ser *embarrassingly parallel*, sus hilos apenas comparten estado, así que no sufre ese coste y 3.13t escala con normalidad. Ahí el único matiz es una ligera regresión de 3.14t a 16 y 32 hilos, que 3.15t corrige, de modo que 3.15t vuelve a ser la versión más rápida.

La causa está en el coste de coordinación del intérprete sin GIL. Cuando el bucle caliente manipula objetos Python desde muchos hilos, la primera build *free-threaded* paga un sobre-coste alto de gestión de referencias y contención de memoria que 3.14t y 3.15t redujeron. EP no lo nota porque casi no comparte datos entre hilos. Este es, con diferencia, el mayor efecto de versión de todo el estudio, y vive por entero en la ejecución interpretada. La sección siguiente comprueba que, al activar el tipado de los modos compilados, esta diferencia desaparece.

Modo 1 interpretado, clase S: tiempo por número de hilos

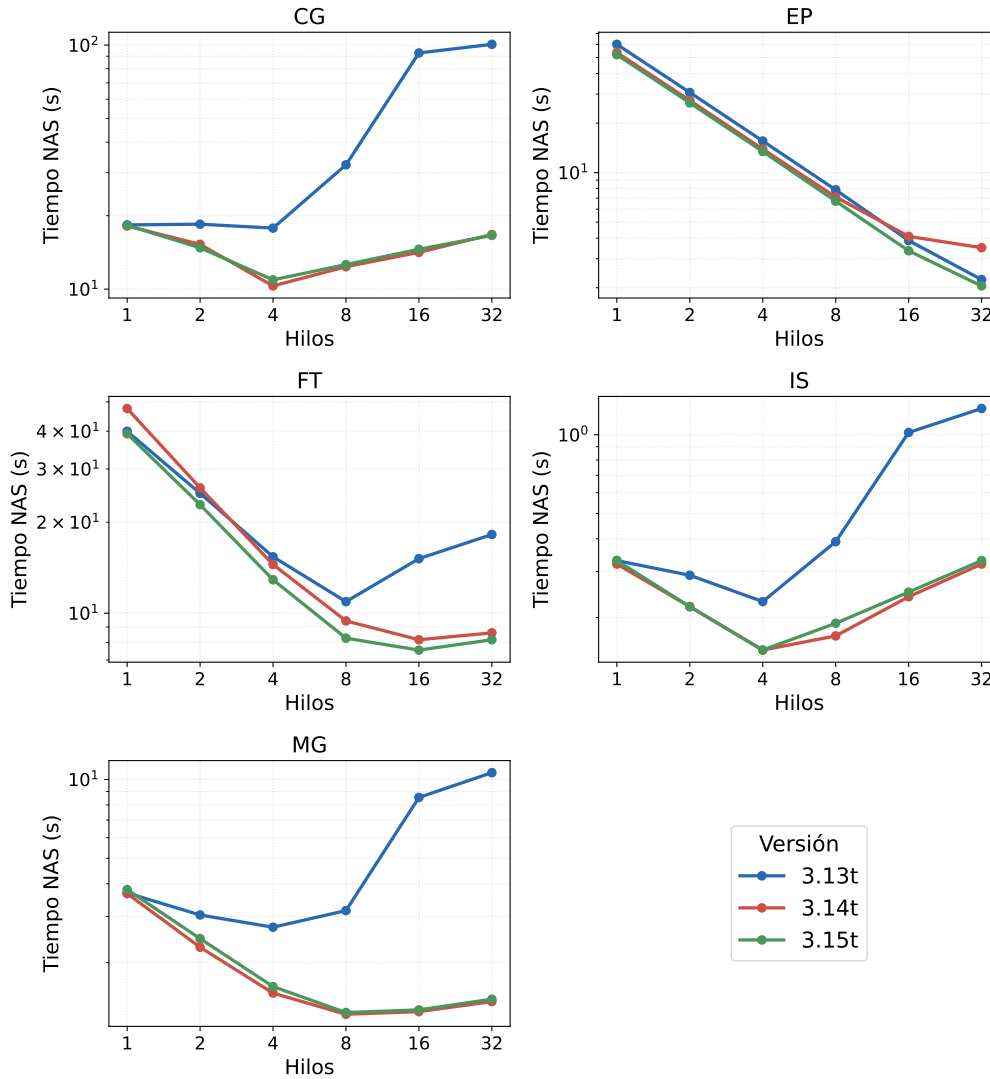


Figura 7.1: Tiempo NAS por número de hilos en modo 1 interpretado, clase S, para Python 3.13t, 3.14t y 3.15t. En CG, FT, IS y MG la 3.13t se dispara al aumentar los hilos, mientras que EP, *embarrassingly parallel*, mantiene las tres versiones juntas.

7.3. El tipado neutraliza el efecto de versión

La pregunta natural es si esa fuerte dependencia de versión sobrevive al tipado. La respuesta es que no. En los modos compilados el bucle caliente deja de manipular objetos Python y pasa a código Cython nativo, de modo que la contención que penalizaba a la 3.13t desaparece.

El contraste es inmediato. En CG clase S a 32 hilos, la ejecución interpretada daba unos 100 s en 3.13t frente a 16 s en 3.15t, mientras que en modo 3 las tres versiones quedan en torno a 0.9 s. El mismo kernel que separaba a las versiones por un factor de seis pasa a ejecutarse en un tiempo casi idéntico en las tres. El efecto de versión no se atenúa, se anula.

Clase	Bench.	Modo 2 (s)			Modo 3 (s)		
		3.13t	3.14t	3.15t	3.13t	3.14t	3.15t
S	CG	14.27	15.31	16.16	0.12	0.12	0.13
S	EP	36.60	42.45	42.24	1.43	1.43	1.44
S	FT	35.48	41.65	36.27	2.30	2.43	2.42
S	IS	0.29	0.30	0.30	0.00	0.00	0.00
S	MG	2.83	3.12	3.24	0.01	0.01	0.01
W	CG	91.32	98.84	105.6	0.76	0.76	0.78
W	EP	73.08	83.94	84.76	2.86	2.87	2.87
W	FT	82.84	93.36	80.89	4.54	4.80	4.86
W	IS	4.83	4.98	5.40	0.03	0.03	0.03
W	MG	170.8	188.0	194.5	0.44	0.62	0.45

Tabla 7.2: Tiempo NAS con un hilo en los modos compilados 2 y 3, clases S y W, para Python 3.13t, 3.14t y 3.15t. El paso de modo 2 a modo 3 (tipado) acelera entre $15\times$ y más de $400\times$ según el benchmark, de forma casi idéntica en las tres versiones.

El tipado domina cualquier diferencia de versión. La Tabla 7.2 compara modo 2 y modo 3 con un hilo. El modo 3 mejora al modo 2 entre $15\times$ y más de $400\times$ según el benchmark, y ese factor es prácticamente el mismo en Python 3.13t, 3.14t y 3.15t. El salto de rendimiento relevante lo da el tipado, no la versión del intérprete.

Ya en modo 3, las tres versiones se comportan igual en todo el rango de hilos. La Figura 7.2 sigue el tiempo NAS por número de hilos en cada benchmark, y las tres curvas se superponen casi exactamente. Sea cual sea la forma de la curva, la fija la estructura del kernel y no la versión del intérprete. El Anexo E recoge los tiempos y speedups completos por número de hilos, los óptimos de cada combinación y las curvas por benchmark¹

¹Los tiempos absolutos de esta campaña pueden diferir ligeramente de los del Capítulo 5 para la misma combinación. Son campañas independientes y la diferencia refleja la variabilidad entre ejecuciones del clúster.

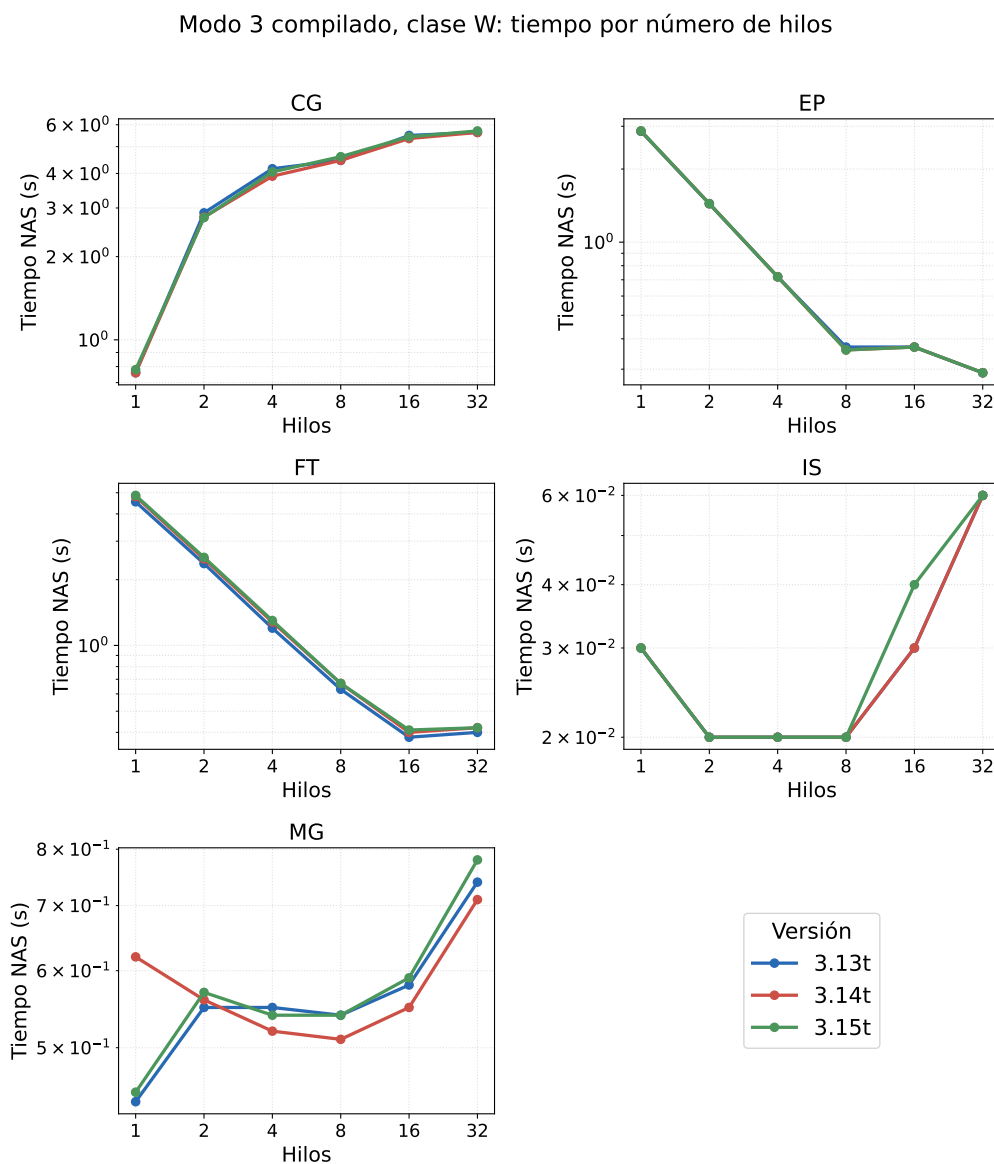


Figura 7.2: Tiempo NAS por número de hilos en modo 3 compilado, clase W, para Python 3.13t, 3.14t y 3.15t. Las tres versiones se superponen en todos los benchmarks, en claro contraste con la divergencia del modo interpretado (Figura 7.1).

7.4. Conclusiones

La comparación entre versiones tiene dos caras. En los modos compilados, que son los que dan tiempos prácticos, el comportamiento final no depende de la versión. Con la afinidad corregida, Python 3.13t, 3.14t y 3.15t mantienen las mismas conclusiones cualitativas, con EP y FT escalando y CG, IS y MG sin escalabilidad sostenida. En los modos interpretados, en cambio, la versión sí importa mucho. Python 3.13t escala bastante peor al aumentar los hilos en CG, FT, IS y MG, y solo EP queda al margen. Esa penalización se anula al tipar. Donde la versión influye, Python 3.15t es siempre la más rápida o queda a la par. Las diferencias prácticas del trabajo están en el modo de ejecución, la estructura del benchmark y la configuración del runtime.

Conclusiones

ESTE trabajo ha adaptado cinco NAS Parallel Benchmarks al modelo de programación OMP4Py y los ha evaluado experimentalmente en Finisterrae III con intérpretes Python *free-threaded*. La evaluación cubre los benchmarks CG, EP, FT, IS y MG en las clases de problema S, W y A, con los cuatro modos de ejecución cuando el coste lo permite y entre 1 y 32 hilos. La campaña principal con Python 3.15.0b1t incluye 630 ejecuciones. La campaña comparativa entre Python 3.13.13t, 3.14.4t y 3.15.0b1t añade 2340 ejecuciones, que cubren tanto los modos interpretados como los compilados. Todas las ejecuciones finales terminaron con verificación SUCCESSFUL.

8.1. Hallazgos principales

8.1.1. Cobertura funcional del port

El port es funcional y cubre los caminos relevantes de OMP4Py. La clase S se ejecutó con los modos 0, 1, 2 y 3, validando los caminos interpretados y compilados. La clase W se evaluó con los modos compilados 2 y 3, y la clase A con el modo 3. Esta estructura permite comprobar corrección sin dedicar recursos a configuraciones que ya son varios órdenes de magnitud más lentas en clases menores.

8.1.2. El tipado Cython como factor dominante

El tipado Cython es el factor de rendimiento dominante. En las clases S y W, el modo 3 mejora al modo 2 en todos los benchmarks medidos, con factores que van desde el orden de $15\times$ en FT hasta más de dos órdenes de magnitud en los casos más favorables. Por tanto, para estos benchmarks el tipado no es una optimización menor. Es la condición que convierte el código OMP4Py en una ejecución competitiva.

8.1.3. Escalabilidad de EP y FT

La campaña principal obtiene speedups altos con Python 3.15.0b1t en los benchmarks con estructura más favorable. En clase A, EP y FT superan $10\times$ de speedup, y FT alcanza su mejor punto con 16 hilos. Estos resultados muestran que el port puede explotar paralelismo real cuando el kernel tiene suficiente trabajo independiente. EP en modo 1 confirma además que la eliminación del GIL produce paralelismo real al nivel del propio Python, aunque esa ruta no sea competitiva en tiempo absoluto frente al modo tipado.

8.1.4. Límites de CG, IS y MG

CG, IS y MG no obtienen escalabilidad sostenida en modo 3. CG empeora al aumentar hilos por el coste de reducciones, normas y barreras frecuentes. IS solo mejora de forma moderada con pocos hilos y pierde eficiencia a 32 hilos. MG combina accesos intensivos a memoria, niveles de malla de distinta granularidad y sincronizaciones entre fases. En estos benchmarks, el tipado mejora el tiempo absoluto, pero no basta para convertir más hilos en speedup.

8.1.5. El tipado neutraliza la versión de Python

En los modos compilados, Python 3.13t, 3.14t y 3.15t escalan casi igual, así que la versión no cambia la conclusión práctica. En los modos interpretados el efecto sí es grande. Python 3.13t escala mucho peor al aumentar los hilos en CG, FT, IS y MG, hasta varias veces más lento a 32 hilos, y solo EP queda al margen por no compartir apenas estado entre hilos. El tipado del modo 3 elimina esa penalización, porque saca el bucle caliente del espacio de objetos Python. Donde la versión influye, Python 3.15t es la más rápida. Como los modos compilados son los únicos con tiempos prácticos, esta diferencia no cambia la conclusión principal del trabajo, pero confirma la mejora de escalado de las builds *free-threaded* posteriores a la 3.13t.

8.1.6. La afinidad del entorno como factor crítico

La afinidad del entorno fue un factor experimental crítico, aunque OMP4Py no implementa la fijación de hilos a núcleos. Las ejecuciones preliminares heredaban variables de afinidad de OpenMP del entorno, y eso producía una serialización aparente en Python 3.15t. El mecanismo, detallado en la Sección 6.2, es indirecto. NumPy carga libgomp, que ante esas variables restringe la máscara de CPU del proceso a un solo núcleo durante el import, y los hilos *free-threaded* heredan esa máscara. Al eliminar OMP_PROC_BIND, OMP_PLACES y GOMP_CPU_AFFINITY por defecto en las campañas finales, EP y FT pasaron a escalar correctamente. Esta corrección cambia la interpretación del trabajo. El problema inicial no era una incapacidad de Python 3.15t ni un fallo del port, sino una restricción de la planificación impuesta desde fuera de la biblioteca.

8.1.7. Escalabilidad real frente a paralelismo ineficiente

El profiling distingue la escalabilidad real del paralelismo ineficiente. Las mediciones con `perf stat` muestran que EP y FT usan varias CPUs y reducen el tiempo. En FT, el IPC y los fallos de caché explican que el óptimo aparezca en 16 hilos en lugar de 32. En CG y MG, el proceso también puede usar varias CPUs, pero el IPC cae y el tiempo empeora. Por tanto, los malos speedups no tienen una única causa. En unos casos el kernel escala, y en otros el paralelismo existe pero la sincronización o la memoria dominan. El sistema de muestreo de Python 3.15 cuantifica ese coste desde el nivel del intérprete. La fracción de tiempo paralelo en espera de barreras y reducciones crece con el número de hilos en los puntos medidos, y muestra que CG, FT, IS y MG crean y destruyen equipos entre regiones paralelas, frente al equipo persistente de EP. El trazado de Python 3.15 en Finisterrae III confirma el gradiente estructural. EP crea un único equipo, mientras que CG crea miles de hilos y decenas de miles de entradas en barrera en una ejecución de clase S.

8.2. Limitaciones

Las amenazas a la validez se agrupan en la Sección 4.7 y se concretan después en la Sección 5.7. La primera limitación es experimental. La comparación entre versiones no incluye clase A, una decisión deliberada para contener el uso del cluster. La segunda es de medida. La salida NAS redondea a dos decimales, de modo que algunos tiempos de clase S en IS y MG sirven para confirmar corrección y orden de magnitud, pero no para ratios finos. La tercera afecta al profiling. `perf stat` mide el proceso completo, y el profiler de Python 3.15 se usa como diagnóstico estructural en modo 1 sobre clase S. El trazado completa los cinco benchmarks y el muestreo aporta los puntos válidos de la campaña. La cuarta es de alcance. Los resultados caracterizan los cinco benchmarks adaptados, los nodos `clk` y el rango de 1 a 32 hilos, pero no agotan otros tamaños, arquitecturas ni políticas de afinidad.

8.3. Trabajo futuro

8.3.1. Instrumentación y campañas complementarias

La prioridad inmediata es cerrar la brecha entre diagnóstico y medición final. El muestreo y el trazado de Python 3.15 ya identifican espera en barreras, reducciones y creación de equipos, pero lo hacen desde fuera del runtime y sobre modo 1. La siguiente campaña debería instrumentar OMP4Py por región paralela y en los modos compilados, midiendo directamente tiempo útil, espera, reducciones y creación de hilos. También convendría extender el mues-

treo a una matriz más fina y a modos compilados, para ampliar la cobertura de plataforma, modo y herramienta.

También queda una campaña selectiva de clase A para versiones. No es necesario repetir toda la matriz. Bastaría con EP y FT, modo 3, Python 3.13t y 3.14t, para confirmar si los *speedups* de clase W se mantienen con la carga mayor. Una segunda campaña fina alrededor de los óptimos de FT e IS permitiría localizar mejor el mejor número de hilos que el barrido por potencias de dos.

8.3.2. Mejoras del runtime

El candidato principal de mejora es reutilizar el equipo de hilos. El runtime actual crea y destruye un equipo por región paralela, y en kernels con muchas regiones cortas, como CG o MG, ese coste aparece miles de veces. Un pool persistente eliminaría parte de ese sobrecoste sin obligar al usuario a envolver manualmente el cómputo en una región paralela grande.

La segunda línea es aligerar barreras y reducciones. La espera medida en `threading.Condition.wait` llega a porcentajes relevantes con varios hilos, y cada reducción escalar añade sincronización. Una barrera más ligera en el runtime nativo y reducciones que eviten barreras completas reducirían el coste de coordinación que limita CG, FT y MG.

8.3.3. Mejoras por benchmark

Las actuaciones con más probabilidad de impacto no son cambiar únicamente de versión de Python ni aumentar hilos sin modificar el runtime. La comparación entre 3.13t, 3.14t y 3.15t muestra curvas muy parecidas cuando la afinidad está corregida. En CG, la línea útil es agrupar más trabajo entre reducciones y optimizar barreras. En IS, paralelizar mejor los prefijos acumulados y reducir el coste de los histogramas por hilo. En MG, adaptar la distribución de trabajo al tamaño de cada nivel de malla para no activar equipos grandes en niveles con muy pocas iteraciones útiles. En todos los casos, mantener los bucles críticos completamente compilables por Cython sigue siendo una condición necesaria.

8.3.4. Extensión a BT, SP y LU

Completar la cobertura con BT, SP y LU permitiría evaluar patrones con dependencias espaciales más complejas. BT y SP son paralelizables con `parallel for` y quedaron fuera por tiempo. LU requiere más que un port directo. Su barrido SSOR necesita sincronización en *wavefront*, que en OpenMP se expresa con `ordered`, `flush`, `atomic` o dependencias entre tareas. Ninguno de esos mecanismos está hoy disponible en las directivas usadas de OMP4Py (Sección 3.1). Incorporar alguno de ellos sería una mejora de la biblioteca, no solo una extensión de la campaña de benchmarks.

8.4. Valoración global

OMP4Py permite expresar paralelismo de estilo OpenMP en Python y producir ejecuciones correctas sobre benchmarks numéricos no triviales. El resultado más sólido es la combinación de corrección funcional y mejora por tipado. El modo 3 reduce los tiempos en más de dos órdenes de magnitud en los casos más favorables y hace viables cargas que en modo 2 serían demasiado costosas.

El rendimiento paralelo es positivo, pero selectivo. EP y FT muestran speedups altos con Python 3.15t y rinden parecido en Python 3.13t y Python 3.14t. CG, IS y MG no escalan bien, ni siquiera con el código tipado y verificado. La utilidad práctica de OMP4Py no es uniforme. Depende de la combinación entre tipado Cython, estructura del kernel, configuración de afinidad y eficiencia del runtime.

El valor principal del trabajo es haber portado benchmarks completos, medido sus caminos de ejecución y separado varios efectos que podrían confundirse, corrección funcional, mejora por compilación, mejora por tipado, escalabilidad real, problemas de configuración de entorno y límites propios de cada benchmark. Esa separación deja una base experimental clara para continuar el desarrollo de OMP4Py y para evaluar futuras versiones de Python *free-threaded*. La contribución central no es afirmar que OMP4Py escala siempre, sino delimitar cuándo escala, cuándo no, y qué parte corresponde al runtime.

Apéndice

Interfaz de ejecución de los benchmarks

Todos los benchmarks adaptados comparten la misma interfaz de línea de comandos, lo que permite reutilizar la infraestructura de campaña sin cambios por benchmark. La invocación general es la siguiente.

```
python <BENCHMARK>_Python.py -c <CLASE> -t <HILOS> -m <MODOS>
```

donde -c selecciona la clase NAS (S, W, A), -t fija el número de hilos del equipo OpenMP y -m selecciona el modo de ejecución de OMP4Py (0 puro, 1 híbrido, 2 compilado, 3 compilado tipado). La salida estándar incluye el tiempo NAS interno, los Mop/s y el estado de verificación (SUCCESSFUL o UNSUCCESSFUL), que son los valores que recopila la infraestructura de medición descrita en el Capítulo 4.

Kernel EP adaptado en modo tipado

El siguiente extracto corresponde al kernel principal de EP en su versión tipada (modo 3), `ep_compute_types`. Ilustra las decisiones de implementación descritas en la Sección 3.3, como la región parallel con reducción sobre `sx` y `sy`, la lista explícita de variables privadas, las `memoryviews` tipadas (`cython.double[:]`) y la sección crítica final que acumula el histograma. Se ha abreviado el cuerpo aritmético del bucle interno por claridad.

```
@omp(compile=use__compiled_types())
def ep_compute_types(nn: int, nk: int, nk_plus: int, nq: int, an: float):
    sx: float = 0.0
    sy: float = 0.0
    ...
    q_values = numpy.repeat(0.0, nq)
    q: cython.double[:] = q_values

    with omp("parallel reduction(+:sx,sy) ")
        "private(i,k,seed_iter,ik,kk,l,a1,x1_seed,t2_int,t4_int,"
        "t1,t3,a2,x2_seed,z,t1_local,t2_local,x1,x2,"
        "t1_value,t2_value,t3_value,t4_value)":
        qq_values = numpy.repeat(0.0, nq)
        x_values = numpy.empty(nk_plus, dtype=numpy.float64)
        qq: cython.double[:] = qq_values
        x_local: cython.double[:] = x_values

    with omp("for schedule(static)"):
        for k in range(1, nn + 1):
            ... # generacion de muestras (Box-Muller)
            for i in range(nk):
                x1 = 2.0 * x_local[2 * i] - 1.0
                x2 = 2.0 * x_local[2 * i + 1] - 1.0
                t1_value = x1 * x1 + x2 * x2
```

```
if t1_value <= 1.0:
    t2_value = math.sqrt(-2.0 * math.log(t1_value)
                       / t1_value)
    t3_value = x1 * t2_value
    t4_value = x2 * t2_value
    l = int(max(abs(t3_value), abs(t4_value)))
    qq[l] = qq[l] + 1.0
    sx = sx + t3_value
    sy = sy + t4_value

with omp("critical"):
    for i in range(nq):
        q[i] = q[i] + qq[i]
```

La declaración explícita de variables privadas en la cláusula `private(...)` es la corrección descrita en la Sección 3.3. Sin ella, varias temporales se compartían entre hilos y la verificación NAS fallaba.

Infraestructura de ejecución en Finisterrae III

Todo el código de este trabajo está disponible en el repositorio público https://github.com/alexxvedo/omp4py_TFM, que reúne los cinco benchmarks adaptados, el runtime de OMP4Py empleado y los scripts de campaña descritos en este anexo. La biblioteca OMP4Py original se mantiene en <https://github.com/citiususc/omp4py>.

La campaña se automatiza con los scripts del directorio `scripts/ft3`. El script `array_run.sbatch` es la plantilla de Slurm que ejecuta cada elemento del array de trabajos. Cada elemento procesa varias filas de la matriz de combinaciones de forma secuencial mediante `run_one.py`.

```
#!/usr/bin/env bash
#SBATCH --job-name=omp4py-npb
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=32
#SBATCH --mem-per-cpu=4G
#SBATCH --time=06:00:00

source "$FT3_ROOT/scripts/ft3/env.sh"
ft3_export_python_runtime "${RUNNER_PYTHON_TAG:-3.15t}"
RUNNER_PYTHON="$(ft3_venv_python "${RUNNER_PYTHON_TAG:-3.15t}")"

start_index=$(( (SLURM_ARRAY_TASK_ID - 1) * rows_per_task + 1 ))
end_index=$(( SLURM_ARRAY_TASK_ID * rows_per_task ))
for index in $(seq "$start_index" "$end_index"); do
    "$RUNNER_PYTHON" "$FT3_ROOT/scripts/ft3/run_one.py" \
        --manifest "$MANIFEST" --index "$index" \
        --campaign-dir "$CAMPAIGN_DIR"
```

done

La selección del intérprete *free-threaded* y la desactivación explícita del GIL se realizan en `env.sh` mediante la variable `PYTHON_GIL=0` y la ruta de bibliotecas correspondiente a cada versión.

```
ft3_export_python_runtime() {  
    local prefix; prefix="$(ft3_python_prefix "$1")"  
    export LD_LIBRARY_PATH=\  
"$prefix/lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}"  
    export PYTHON_GIL=0  
}
```

Estructura de los resultados de campaña

Cada campaña genera un directorio bajo `results/` con los siguientes artefactos, que constituyen la base de las tablas y figuras de los Capítulos 5, 6 y 7.

- `manifest.csv`: matriz completa de combinaciones de la campaña.
- `raw/`: registros crudos con nombre semántico (versión de Python, benchmark, clase, modo, número de hilos y repetición).
- `records/`: un fichero JSON por ejecución, con tiempo NAS interno, tiempo externo de proceso, Mop/s, código de retorno y verificación.
- `summary.csv`: tabla plana con todas las ejecuciones.
- `summary_best.csv`: agregados de mejor y media por combinación.
- `env.txt`: metadatos de la campaña (versiones de Python, NumPy, Cython y variables de entorno).

La relación entre artefactos y uso dentro de la memoria es la siguiente.

Artefacto	Uso principal
manifest.csv	Define la matriz experimental con benchmark, clase, modo, versión de Python, hilos y repetición. Permite comprobar que las tablas proceden de una campaña cerrada y no de ejecuciones aisladas.
records/ y raw/	Conservan la evidencia por ejecución, con salida NAS, verificación, código de retorno, tiempo interno y tiempo externo. Son la base auditable de los Capítulos 5, 6 y 7.
summary.csv	Alimenta las tablas completas y permite recalcular medias, mejores tiempos, speedups y comprobaciones de verificación.
summary_best.csv	Fuente directa de las tablas resumen de tiempos, speedups, óptimos y ganadores por versión.
env.txt	Documenta intérprete, bibliotecas y variables de entorno, incluida la afinidad, para interpretar la validez de la campaña.
py315_profile_summary.csv, py315_sampling_summary.csv y py315_tracing_summary.csv	Resúmenes del profiling con Python 3.15. De ellos se derivan las tablas de muestreo, gestión de hilos y trazado del Capítulo 6.
latex/	Tablas generadas automáticamente a partir de los resúmenes, incluidas las del muestreo y el trazado de Python 3.15 en Finisterrae III.

El registro de la versión exacta de cada componente y de las variables de entorno responde a la implicación metodológica del Capítulo 7. En evaluaciones de Python *free-threaded* la configuración del runtime y la afinidad del entorno pueden alterar la conclusión experimental, por lo que deben quedar documentadas junto con los resultados.

Material completo de la comparación entre versiones en modo 3

El Capítulo 7 muestra que, en los modos compilados, las tres versiones de Python se comportan de forma muy parecida, por lo que el cuerpo se queda con una figura resumen. Este anexo reúne el material de modo 3 que respalda esa síntesis. Las dos primeras tablas dan los tiempos y speedups completos para todos los recuentos de hilos. Después vienen los óptimos por combinación, la versión ganadora a 32 hilos, el tiempo absoluto a 32 hilos y las curvas de speedup por benchmark, donde se aprecia que las tres versiones siguen trazas casi idénticas.

Clase	Bench.	Python	1	2	4	8	16	32
S	CG	3.13t	0.12	0.48	0.65	0.72	0.86	0.91
S	CG	3.14t	0.12	0.47	0.62	0.71	0.86	0.92
S	CG	3.15t	0.13	0.46	0.61	0.70	0.86	0.92
S	EP	3.13t	1.43	0.72	0.36	0.19	0.19	0.15
S	EP	3.14t	1.43	0.72	0.36	0.18	0.19	0.15
S	EP	3.15t	1.44	0.72	0.36	0.18	0.19	0.16
S	FT	3.13t	2.30	1.19	0.62	0.33	0.20	0.22
S	FT	3.14t	2.43	1.25	0.65	0.35	0.22	0.23
S	FT	3.15t	2.42	1.26	0.66	0.35	0.22	0.24
S	IS	3.13t	0.00	0.00	0.00	0.01	0.01	0.02
S	IS	3.14t	0.00	0.00	0.00	0.01	0.01	0.02
S	IS	3.15t	0.00	0.00	0.00	0.01	0.01	0.02
S	MG	3.13t	0.01	0.02	0.03	0.05	0.10	0.21
S	MG	3.14t	0.01	0.02	0.03	0.05	0.10	0.21
S	MG	3.15t	0.01	0.02	0.03	0.06	0.10	0.22
W	CG	3.13t	0.76	2.88	4.16	4.46	5.49	5.63
W	CG	3.14t	0.76	2.78	3.91	4.46	5.35	5.63
W	CG	3.15t	0.78	2.77	4.05	4.60	5.43	5.70
W	EP	3.13t	2.86	1.44	0.72	0.37	0.37	0.29
W	EP	3.14t	2.87	1.44	0.72	0.36	0.37	0.29
W	EP	3.15t	2.87	1.44	0.72	0.36	0.37	0.29
W	FT	3.13t	4.54	2.37	1.20	0.63	0.38	0.40
W	FT	3.14t	4.80	2.49	1.28	0.67	0.40	0.42
W	FT	3.15t	4.86	2.53	1.30	0.67	0.41	0.42
W	IS	3.13t	0.03	0.02	0.02	0.02	0.03	0.06
W	IS	3.14t	0.03	0.02	0.02	0.02	0.03	0.06
W	IS	3.15t	0.03	0.02	0.02	0.02	0.04	0.06
W	MG	3.13t	0.44	0.55	0.55	0.54	0.58	0.74
W	MG	3.14t	0.62	0.56	0.52	0.51	0.55	0.71
W	MG	3.15t	0.45	0.57	0.54	0.54	0.59	0.78

Tabla E.1: Mejor tiempo NAS por número de hilos en modo 3 para cada versión de Python.

Clase	Bench.	Python	1	2	4	8	16	32
S	CG	3.13t	1.00×	0.25×	0.18×	0.17×	0.14×	0.13×
S	CG	3.14t	1.00×	0.26×	0.19×	0.17×	0.14×	0.13×
S	CG	3.15t	1.00×	0.28×	0.21×	0.19×	0.15×	0.14×
S	EP	3.13t	1.00×	1.99×	3.97×	7.53×	7.53×	9.53×
S	EP	3.14t	1.00×	1.99×	3.97×	7.94×	7.53×	9.53×
S	EP	3.15t	1.00×	2.00×	4.00×	8.00×	7.58×	9.00×
S	FT	3.13t	1.00×	1.93×	3.71×	6.97×	11.50×	10.45×
S	FT	3.14t	1.00×	1.94×	3.74×	6.94×	11.05×	10.57×
S	FT	3.15t	1.00×	1.92×	3.67×	6.91×	11.00×	10.08×
S	IS	3.13t	—	—	—	—	—	—
S	IS	3.14t	—	—	—	—	—	—
S	IS	3.15t	—	—	—	—	—	—
S	MG	3.13t	1.00×	0.50×	0.33×	0.20×	0.10×	0.05×
S	MG	3.14t	1.00×	0.50×	0.33×	0.20×	0.10×	0.05×
S	MG	3.15t	1.00×	0.50×	0.33×	0.17×	0.10×	0.05×
W	CG	3.13t	1.00×	0.26×	0.18×	0.17×	0.14×	0.13×
W	CG	3.14t	1.00×	0.27×	0.19×	0.17×	0.14×	0.13×
W	CG	3.15t	1.00×	0.28×	0.19×	0.17×	0.14×	0.14×
W	EP	3.13t	1.00×	1.99×	3.97×	7.73×	7.73×	9.86×
W	EP	3.14t	1.00×	1.99×	3.99×	7.97×	7.76×	9.90×
W	EP	3.15t	1.00×	1.99×	3.99×	7.97×	7.76×	9.90×
W	FT	3.13t	1.00×	1.92×	3.78×	7.21×	11.95×	11.35×
W	FT	3.14t	1.00×	1.93×	3.75×	7.16×	12.00×	11.43×
W	FT	3.15t	1.00×	1.92×	3.74×	7.25×	11.85×	11.57×
W	IS	3.13t	1.00×	1.50×	1.50×	1.50×	1.00×	0.50×
W	IS	3.14t	1.00×	1.50×	1.50×	1.50×	1.00×	0.50×
W	IS	3.15t	1.00×	1.50×	1.50×	1.50×	0.75×	0.50×
W	MG	3.13t	1.00×	0.80×	0.80×	0.81×	0.76×	0.59×
W	MG	3.14t	1.00×	1.11×	1.19×	1.22×	1.13×	0.87×
W	MG	3.15t	1.00×	0.79×	0.83×	0.83×	0.76×	0.58×

Tabla E.2: Speedup por número de hilos en modo 3 para cada versión de Python.

Clase	Bench.	Python	Mejor hilo	Mejor tiempo (s)	Speedup ópt.	T32 (s)	Speedup32
S	CG	3.13t	1	0.12	1.00×	0.91	0.13×
S	CG	3.14t	1	0.12	1.00×	0.92	0.13×
S	CG	3.15t	1	0.13	1.00×	0.92	0.14×
S	EP	3.13t	32	0.15	9.53×	0.15	9.53×
S	EP	3.14t	32	0.15	9.53×	0.15	9.53×
S	EP	3.15t	32	0.16	9.00×	0.16	9.00×
S	FT	3.13t	16	0.20	11.50×	0.22	10.45×
S	FT	3.14t	16	0.22	11.05×	0.23	10.57×
S	FT	3.15t	16	0.22	11.00×	0.24	10.08×
S	IS	3.13t	1	0.00	—	0.02	0.00×
S	IS	3.14t	1	0.00	—	0.02	0.00×
S	IS	3.15t	1	0.00	—	0.02	0.00×
S	MG	3.13t	1	0.01	1.00×	0.21	0.05×
S	MG	3.14t	1	0.01	1.00×	0.21	0.05×
S	MG	3.15t	1	0.01	1.00×	0.22	0.05×
W	CG	3.13t	1	0.76	1.00×	5.63	0.13×
W	CG	3.14t	1	0.76	1.00×	5.63	0.13×
W	CG	3.15t	1	0.78	1.00×	5.70	0.14×
W	EP	3.13t	32	0.29	9.86×	0.29	9.86×
W	EP	3.14t	32	0.29	9.90×	0.29	9.90×
W	EP	3.15t	32	0.29	9.90×	0.29	9.90×
W	FT	3.13t	16	0.38	11.95×	0.40	11.35×
W	FT	3.14t	16	0.40	12.00×	0.42	11.43×
W	FT	3.15t	16	0.41	11.85×	0.42	11.57×
W	IS	3.13t	2	0.02	1.50×	0.06	0.50×
W	IS	3.14t	2	0.02	1.50×	0.06	0.50×
W	IS	3.15t	2	0.02	1.50×	0.06	0.50×
W	MG	3.13t	1	0.44	1.00×	0.74	0.59×
W	MG	3.14t	8	0.51	1.22×	0.71	0.87×
W	MG	3.15t	1	0.45	1.00×	0.78	0.58×

Tabla E.3: Mejor número de hilos en modo 3 para cada versión, clase y benchmark.

Clase	Bench.	Ganadora	Tiempo (s)	Segunda (s)	Margen
S	CG	3.13t	0.91	0.92	1.10 %
S	EP	3.13t	0.15	0.15	0.00 %
S	FT	3.13t	0.22	0.23	4.55 %
S	IS	3.13t	0.02	0.02	0.00 %
S	MG	3.13t	0.21	0.21	0.00 %
W	CG	3.13t	5.63	5.63	0.00 %
W	EP	3.13t	0.29	0.29	0.00 %
W	FT	3.13t	0.40	0.42	5.00 %
W	IS	3.13t	0.06	0.06	0.00 %
W	MG	3.14t	0.71	0.74	4.23 %

Tabla E.4: Versión más rápida con 32 hilos en modo 3 y margen sobre la segunda mejor.

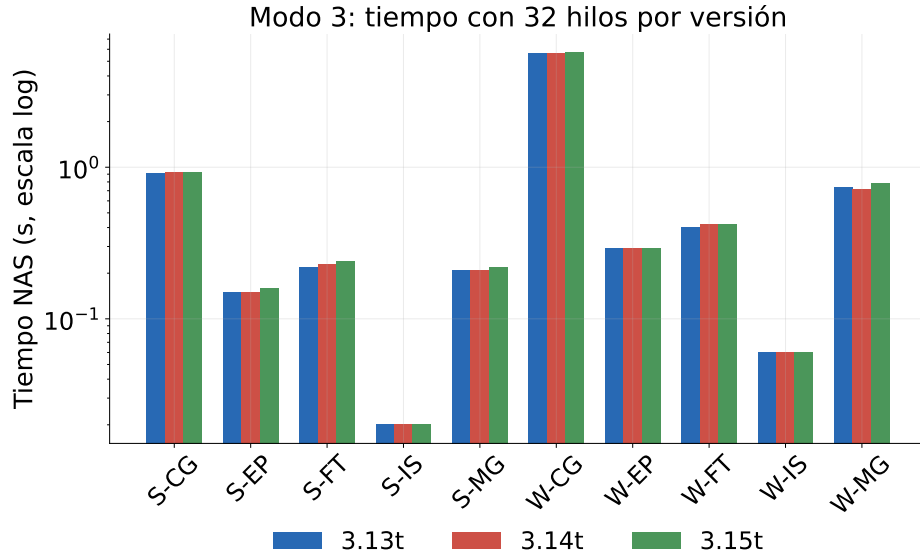


Figura E.1: Tiempo NAS con 32 hilos en modo 3, clases S y W, para las tres versiones de Python. El eje vertical usa escala logarítmica.

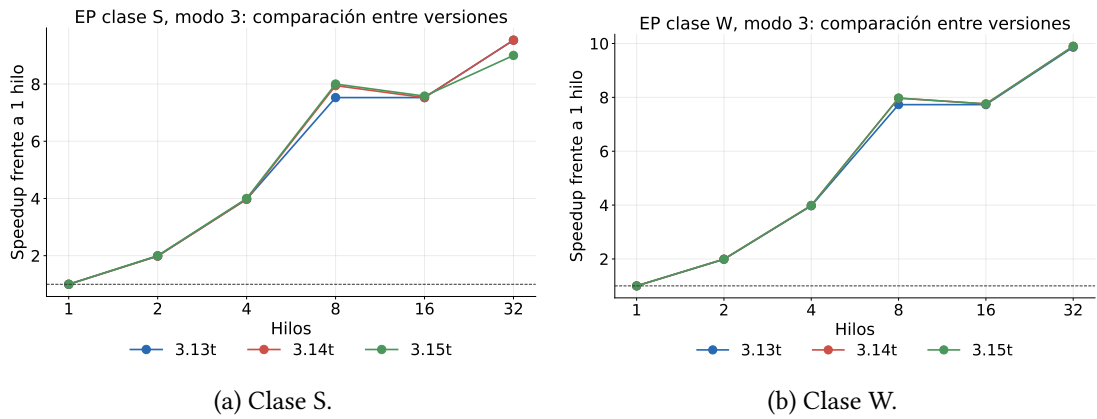
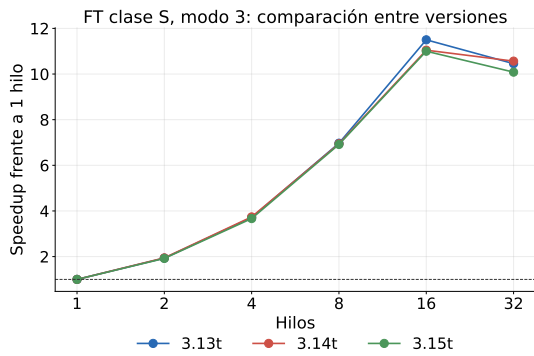
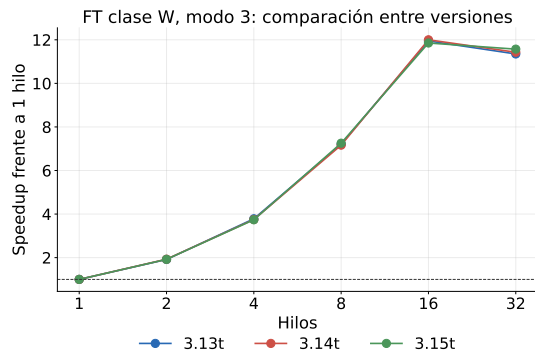


Figura E.2: EP, modo 3. Speedup por versión de Python en las clases S y W.

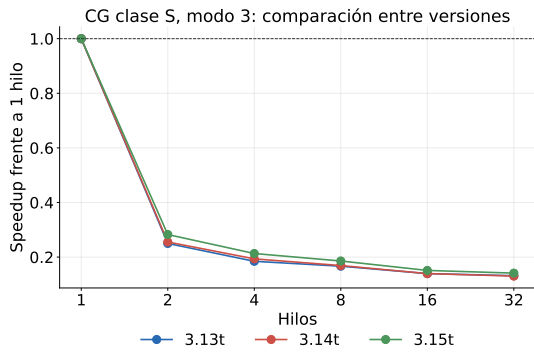


(a) Clase S.

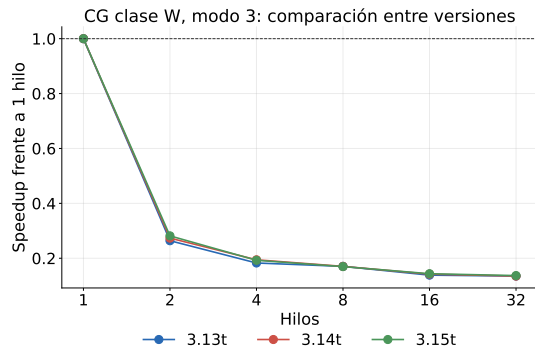


(b) Clase W.

Figura E.3: FT, modo 3. Speedup por versión de Python en las clases S y W.

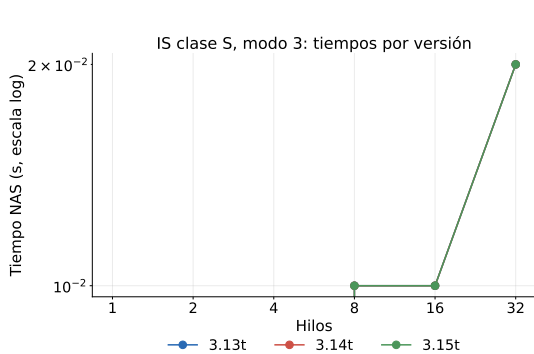


(a) Clase S.

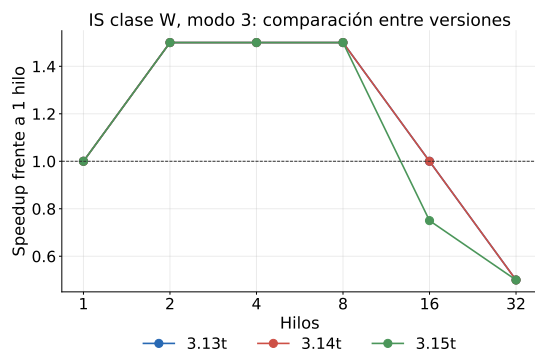


(b) Clase W.

Figura E.4: CG, modo 3. Speedup por versión de Python en las clases S y W.



(a) Clase S (tiempo NAS).



(b) Clase W (speedup).

Figura E.5: IS, modo 3, por versión de Python. En clase S se muestra el tiempo NAS porque el speedup no es fiable (tiempo con un hilo redondeado a 0.00 s). En clase W se muestra el speedup.

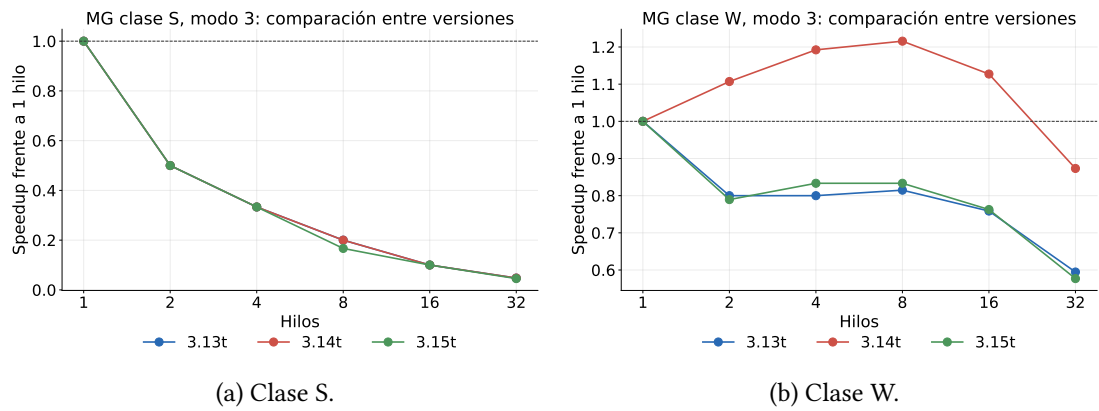


Figura E.6: MG, modo 3. Speedup por versión de Python en las clases S y W.

Glosario de acrónimos

ABI *Application Binary Interface*. Interfaz binaria de aplicación; en este trabajo, la del intérprete CPython compilado en modo *free-threaded*.

API *Application Programming Interface*. Interfaz de programación de aplicaciones.

AST *Abstract Syntax Tree*. Árbol de sintaxis abstracta generado por el analizador sintáctico del intérprete Python a partir del código fuente.

CESGA Centro de Supercomputación de Galicia.

CG *Conjugate Gradient*. Benchmark NAS que evalúa operaciones sobre matrices dispersas mediante el método del gradiente conjugado.

CPU *Central Processing Unit*. Unidad central de procesamiento.

DSL *Domain-Specific Language*. Lenguaje específico de dominio.

EP *Embarrassingly Parallel*. Benchmark NAS que evalúa la generación de pares gaussianos mediante el método de Box-Muller.

FFT *Fast Fourier Transform*. Transformada rápida de Fourier, algoritmo eficiente para calcular la transformada discreta de Fourier.

FT *Fourier Transform*. Benchmark NAS que evalúa la transformada discreta de Fourier tridimensional sobre un array complejo.

FT3 Finisterrae III. Supercomputador del CESGA utilizado para las campañas experimentales.

GIL *Global Interpreter Lock*. Mecanismo de bloqueo global del intérprete CPython que impide la ejecución simultánea de hilos Python sobre el bytecode del intérprete.

HPC *High Performance Computing*. Computación de altas prestaciones.

-
- IPC** *Instructions Per Cycle*. Instrucciones retiradas por ciclo de reloj; métrica de eficiencia de ejecución medida con `perf stat`.
- IS** *Integer Sort*. Benchmark NAS que evalúa la ordenación de enteros mediante un algoritmo de tipo bucket sort.
- MG** *Multi-Grid*. Benchmark NAS que evalúa operadores de restricción, prolongación e interpolación sobre mallas en múltiples niveles.
- Mop/s** Millones de operaciones por segundo. Métrica de rendimiento empleada por los NAS Parallel Benchmarks.
- NAS** *NASA Advanced Supercomputing*. División de la NASA que desarrolló los NAS Parallel Benchmarks.
- NPB** *NAS Parallel Benchmarks*. Conjunto de programas de referencia para la evaluación de sistemas de computación paralela.
- OpenMP** *Open Multi-Processing*. Estándar de programación paralela basado en directivas de compilador para sistemas de memoria compartida.
- PEP** *Python Enhancement Proposal*. Documento de propuesta de mejora del lenguaje Python.

Glosario de términos

Aceleración (*speedup*) Cociente entre el tiempo de ejecución con un hilo y el tiempo con t hilos dentro del mismo modo y clase. Un valor superior a 1 indica ganancia con el paralelismo; un valor inferior a 1 indica degradación.

Barrera (*barrier*) Punto de sincronización que obliga a todos los hilos de una región paralela a esperar hasta que el último de ellos llegue al mismo punto antes de continuar.

Benchmark Programa de referencia diseñado para medir el rendimiento de un sistema de forma reproducible y comparable entre plataformas.

Cython Compilador que acepta un superconjunto del lenguaje Python con anotaciones de tipo opcionales y genera código C que se compila como módulo de extensión para CPython. Las anotaciones de tipo permiten sustituir objetos Python por tipos C nativos en los bucles críticos.

Free-threaded Python Variante experimental de CPython, disponible desde la versión 3.13 mediante la opción `-disable-gil`, que elimina el GIL y permite la ejecución simultánea de múltiples hilos sobre el bytecode del intérprete.

Kernel numérico Función que concentra la mayor parte del cómputo útil de un benchmark o aplicación paralela, separada del código de orquestación (inicialización, E/S, verificación).

Memoryview Objeto Python que expone el buffer interno de un array NumPy sin copiar los datos. En Cython, las memoryviews tipadas permiten acceder directamente a los elementos del array como punteros C, eliminando la indirección de los objetos Python en los bucles internos.

Modo de ejecución En OMP4Py, configuración que determina qué partes del código se compilan: modo 0 (runtime puro de OMP4Py en Python), modo 1 (híbrido, runtime nativo

cuando está disponible; kernels sin compilar), modo 2 (kernels compilados con Cython) y modo 3 (kernels compilados con tipado Cython explícito).

OMP4Py Implementación en Python puro del modelo de programación OpenMP. Traduce directivas OpenMP embebidas como cadenas de texto en código Python paralelo mediante transformación del AST, con soporte opcional de compilación Cython.

Overhead de sincronización Tiempo invertido en operaciones de coordinación entre hilos (barreras, secciones críticas, reducciones) que no contribuye directamente al cómputo útil.

Profiler de muestreo Herramienta que observa periódicamente las pilas de llamadas de los hilos de un proceso. En este trabajo se usa para estimar qué fracción del tiempo paralelo cae en cómputo, espera o inicialización.

Profiler de trazado Herramienta determinista que registra llamadas a funciones y permite obtener recuentos exactos. En este trabajo se usa para contar regiones paralelas, hilos creados y entradas en barrera.

Reducción Operación que combina los valores parciales calculados por cada hilo en un único resultado escalar (suma, máximo, mínimo, etc.) al final de una región paralela.

Región paralela Bloque de código delimitado por una directiva `parallel` de OpenMP que se ejecuta simultáneamente en múltiples hilos.

Slurm Sistema de gestión y planificación de trabajos utilizado en los supercomputadores del CESGA para asignar recursos de cómputo y organizar la ejecución de trabajos en cola.

Tipado Cython Proceso de anotar variables en código Cython con tipos C explícitos (`cython.int`, `cython.double`, etc.) para que el compilador genere operaciones nativas en lugar de pasar por el mecanismo de despacho de objetos Python.

Verificación NAS Comprobación numérica integrada en cada NAS Parallel Benchmark que calcula un checksum de los resultados y lo compara con un valor de referencia para la clase ejecutada. Una verificación **SUCCESSFUL** garantiza la corrección del cómputo.

Bibliografía

- [1] M. Gorelick and I. Ozsvald, *High Performance Python: Practical Performant Programming for Humans*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020.
- [2] S. Gross, “Pep 703 – making the global interpreter lock optional in cpython,” <https://peps.python.org/pep-0703/>, 2023.
- [3] C. Piñeiro and J. C. Pichel, “Omp4py: A pure python implementation of openmp,” *Future Generation Computer Systems*, vol. 175, 2026.
- [4] —, “Unlocking python multithreading capabilities using openmp-based programming with omp4py,” in *IEEE/ACM International Symposium on Code Generation and Optimization*, 2026.
- [5] OpenMP Architecture Review Board, “Openmp application programming interface,” <https://www.openmp.org/specifications/>.
- [6] D. H. Bailey, E. Barszcz, J. T. Barton, D. S. Browning, R. L. Carter, L. Dagum, R. Fatoohi, P. O. Frederickson, T. A. Lasinski, R. S. Schreiber, H. D. Simon, V. Venkatakrisnan, and S. K. Weeratunga, “The nas parallel benchmarks,” NASA Ames Research Center, Tech. Rep. RNR-94-007, 1994.
- [7] G. Hager and G. Wellein, *Introduction to High Performance Computing for Scientists and Engineers*, 1st ed. Boca Raton, FL, USA: CRC Press, Inc., 2010.
- [8] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.
- [9] L. Dagum and R. Menon, “OpenMP: An industry standard API for shared-memory programming,” *IEEE Computational Science and Engineering*, vol. 5, no. 1, pp. 46–55, 1998.

- [10] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn, and K. Smith, “Cython: The best of both worlds,” *Computing in Science & Engineering*, vol. 13, no. 2, pp. 31–39, 2011.
- [11] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A LLVM-based Python JIT compiler,” in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM ’15)*, 2015.
- [12] T. A. Anderson and T. G. Mattson, “Multithreaded parallel Python through OpenMP support in Numba,” in *Proceedings of the 20th Python in Science Conference (SciPy 2021)*, 2021.
- [13] L. D. Dalcín, R. R. Paz, P. A. Kler, and A. Cosimo, “Parallel distributed computing using Python,” *Advances in Water Resources*, vol. 34, no. 9, pp. 1124–1139, 2011.
- [14] M. Rocklin, “Dask: Parallel computation with blocked algorithms and task scheduling,” in *Proceedings of the 14th Python in Science Conference (SciPy)*, 2015, pp. 126–132.
- [15] D. Di Domenico *et al.*, “NPB-Python: NAS Parallel Benchmarks for Python,” <https://github.com/danidomenico/NPB-PYTHON>, 2021.
- [16] Linux perf contributors, “perf-stat(1): Run a command and gather performance counter statistics,” <https://man7.org/linux/man-pages/man1/perf-stat.1.html>, 2026.
- [17] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” *Proceedings of the AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967.
- [18] P. Galindo Salgado *et al.*, “Pep 768 – safe external debugger interface for cpython,” <https://peps.python.org/pep-0768/>, 2025.
- [19] Python Software Foundation, “The Python profilers: Statistical sampling profiler (profiling.sampling),” <https://docs.python.org/3.15/library/profiling.sampling.html>, 2026.